

# Multi-node Training

in K-Fold Project

Hyosoon Jang

2026.05.10

# Outline

1. ~~K-Fold project~~ 5 min
2. ~~Structure representation learning~~ 20 min
3. Multi-node training 50-60min ~~20 min~~

## Preview

# Tough Experience

### [GPU 사용량 모니터링]

이 채널에서 메시지 검색

지난 3일 간 gpu 사용량 로그입니다. 24시간 GPU-tils의 평균입니다. (참고로 gpu28은 )

세 연구실 gpu 할당 현황은 다음과 같습니다:

김우연 교수님 연구실: gpu05~gpu13

황성주 교수님 연구실: gpu14~gpu25

안성수 교수님 연구실: gpu26~gpu32

3일간의 통계로 보았을 때 gpu22 ~ gpu26의 사용량이 10% 내외 정도로 나오고 있습니

할당된 GPU 자원에 상당한 부족을 느끼고 있는 팀들이 있습니다.

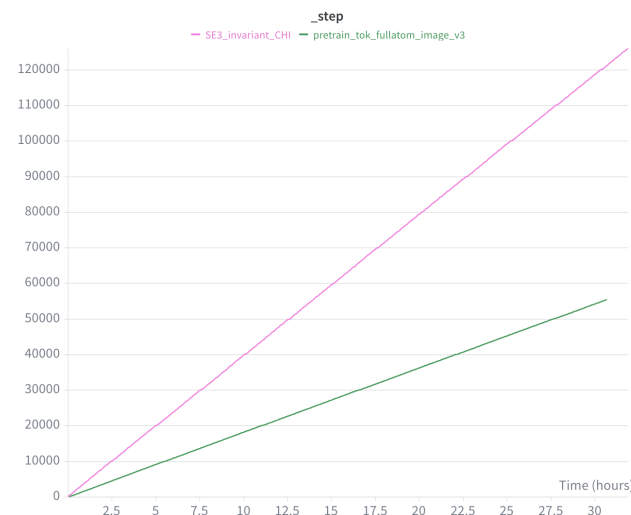
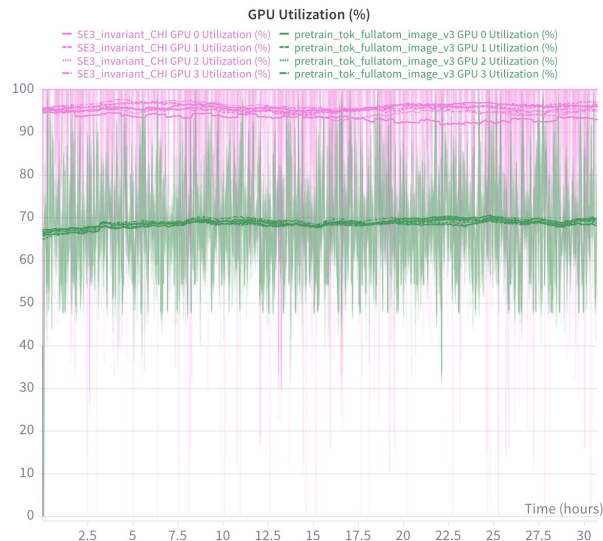
GPU 자원의 탄력적 운용을 위해 해당 gpu들의 향후 사용 계획이 어떻게 되시는지 공유

혹은, 당장 실험 계획이 없으시다면 없다고 말씀주시면 감사하겠습니다. (편집됨)

3개 파일 모든 다운로드

| 12월 02일 — Two-Column 정리 |      | 12월 03일 — Two-Column 정리 |      | 12월 04일 — Two-Column 정리 |   |
|-------------------------|------|-------------------------|------|-------------------------|---|
| GPU                     | %    | GPU                     | %    | GPU                     | % |
| GPU01                   | 0    | GPU17                   | 94   | GPU01                   | 0 |
| GPU02                   | 12.6 | GPU18                   | 10.4 | GPU02                   | 0 |
| GPU03                   | 35.3 | GPU19                   | 11   | GPU03                   | 0 |
| GPU04                   | 45.5 | GPU20                   | 0    | GPU04                   | 0 |
| GPU05                   | 95.5 | GPU21                   | 0    | GPU05                   | 0 |
| GPU06                   | 95.9 | GPU22                   | 18.1 | GPU06                   | 0 |
| GPU07                   | 93.3 | GPU23                   | 0    | GPU07                   | 0 |
| GPU08                   | 93   | GPU24                   | 1    | GPU08                   | 0 |
| GPU09                   | 97   | GPU25                   | 1    | GPU09                   | 0 |
| GPU10                   | 65.1 | GPU26                   | 16.6 | GPU10                   | 0 |
| GPU11                   | 97   | GPU27                   | 75.1 | GPU11                   | 0 |
| GPU12                   | 97   | GPU28                   | 1    | GPU12                   | 0 |
| GPU13                   | 59   | GPU29                   | 62.9 | GPU13                   | 0 |
| GPU14                   | 62.6 | GPU30                   | 80.9 | GPU14                   | 0 |
| GPU15                   | 93.6 | GPU31                   | 35.3 | GPU15                   | 0 |
| GPU16                   | 93.9 | GPU32                   | 29.3 | GPU16                   | 0 |

horrible GPU monitoring



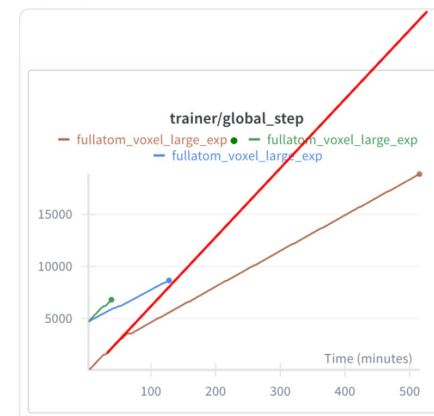
2026/05 vs. 2026/01



Hyosoon Jang 오후 1:43

아까 말씀드린 이슈에 대하여 원래는 빨간 라인 수준으로 찍혀야합니다 (50분에 1epoch. 지금은 100분에 1epoch) (편집됨)

image.png



Hyosoon Jang 1월 13일 오후 6:51

해결했습니다  $\pi\pi$  (이제 50분에 1epoch) (편집됨)

image.png



initial multi-node training

## Overview

# Computation Time



How can we optimize these?



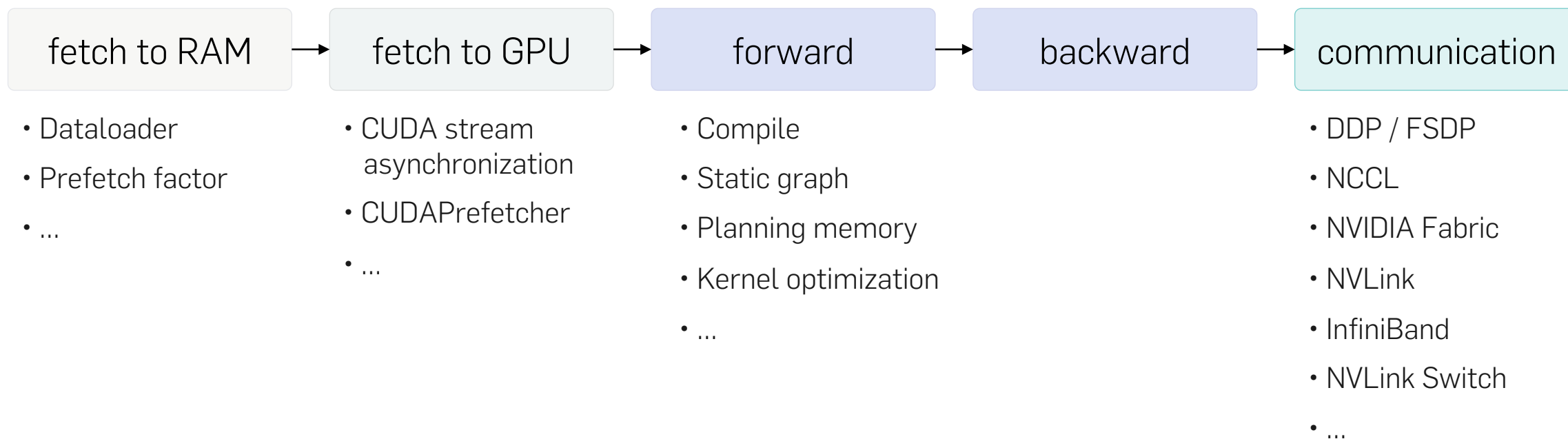
- **Forward and backward computation** time matters in general
- **Communication** overhead matters in **multi-GPU training**
- **Fetch to GPU** time matters when **probing high-dimensional representations** (e.g., pair reps)

## Overview

# Computation Time



How can we optimize these?



## Overview

# Computation Time



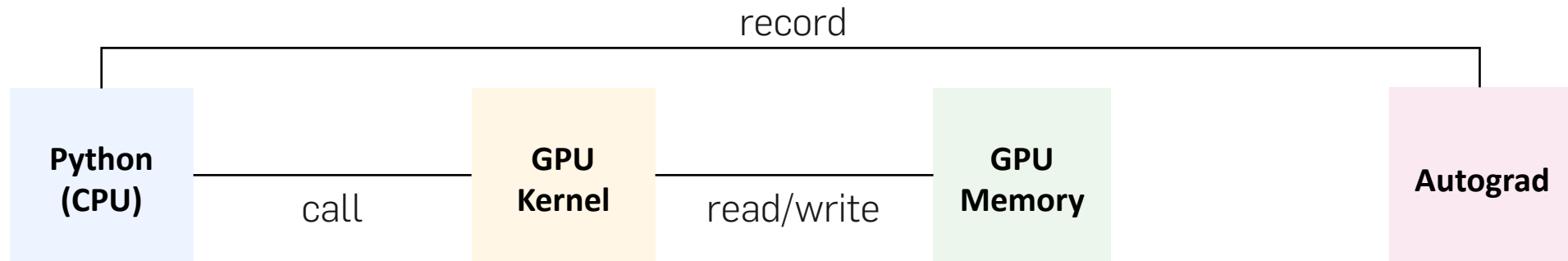
How can we optimize these?



- **Forward and backward computation** time matters in general
- **Synchronization** overhead matters in **multi-node training**
- **Fetch to GPU** time matters when **probing high-dimensional representations** (e.g., pair reps)

## Recap

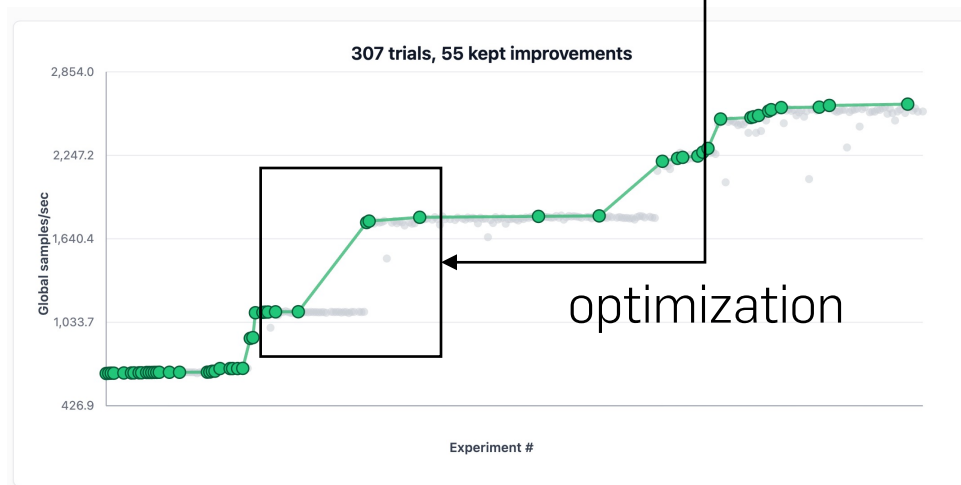
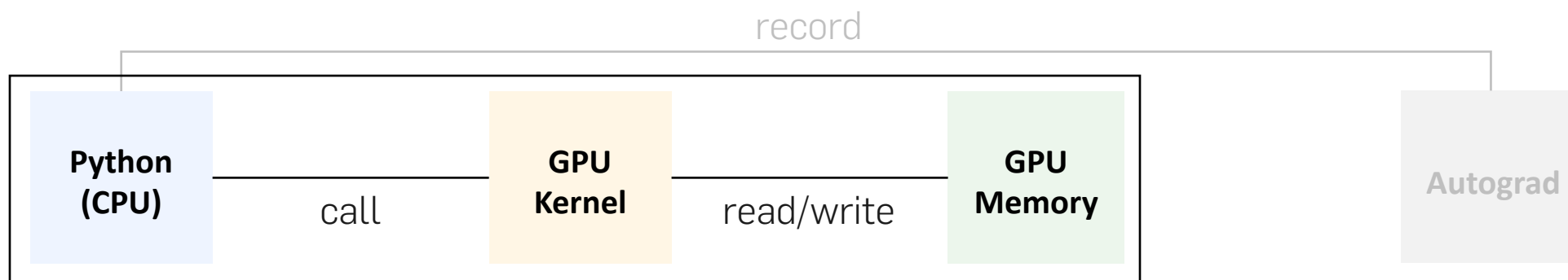
# Forward & Backward Computation



- Python calls a **GPU kernel for each operation** (+, /, -, \*, MatMul) and **records it in Autograd**
- Each kernel **reads/writes GPU memory once**, and performs **computation on GPU cores**  
(e.g.,  $x = a + b - c * d$ : three kernel calls, and three writes to GPU memory ( $[a+b]$ ,  $[c*d]$ ,  $[a+b]-[c*d]$ ))
- Optimization aims to minimize **the number of such interactions**

Recap

# Forward & Backward Computation



**Nayoung Kim** 오후 1:54

이번에 autoresearch 형태로 model throughput optimization 해봤는데, 생각보다 잘돼서 (약 x6) 실험

---

*Experiment summary: CSD training throughput optimization*

*Goal*

Improve MatterFlow CSD training-step throughput without changing model/training semantics

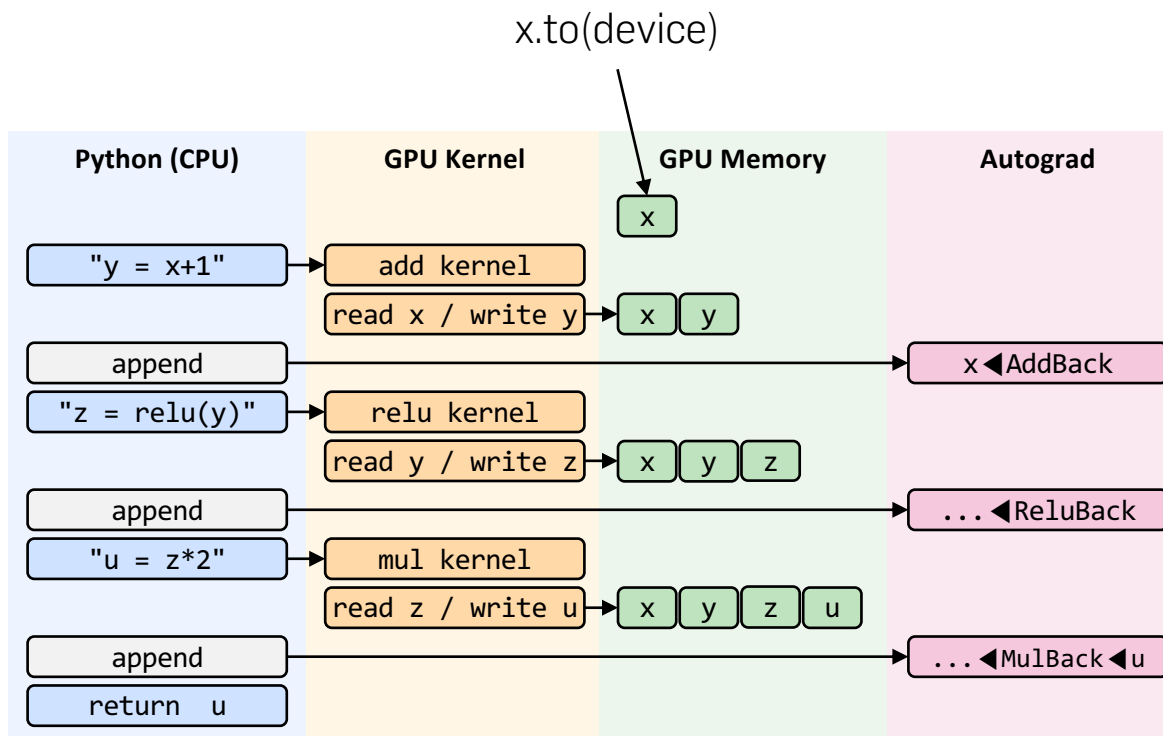
shout out to Nayoung Kim



## Recap

# Forward Computation

```
y = x+1  
z = relu(y)  
u = z*2  
u.backward()
```



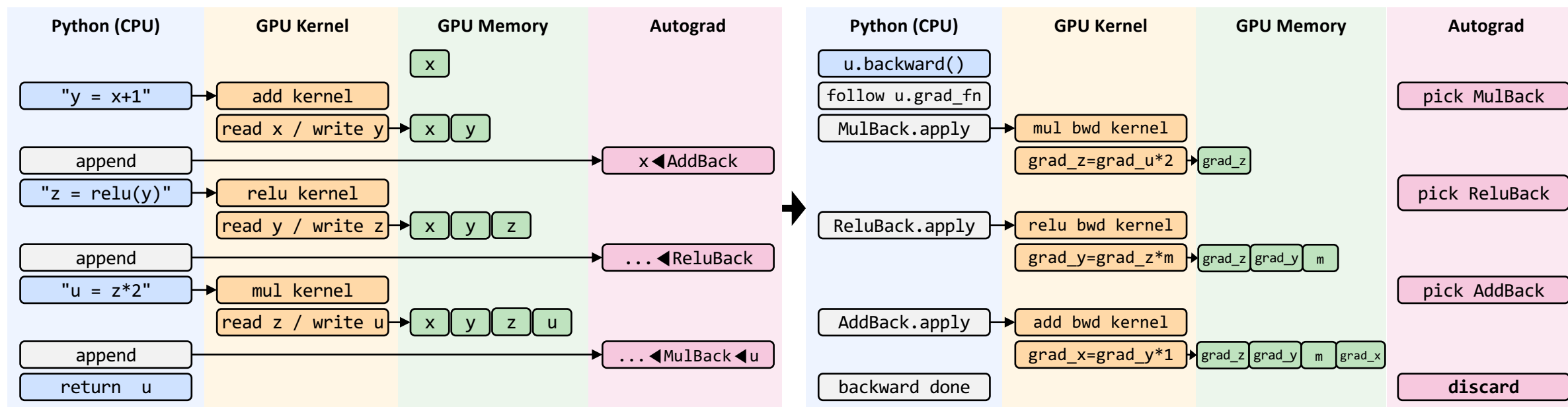
- Each operation code calls **GPU kernel**
- **GPU kernel** reads from and writes to **GPU memory**
- Save **computational graphs** to **Autograd** (details in later)

## Recap

# Backward Computation

```
y = x+1  
z = relu(y)  
u = z*2  
u.backward()
```

- Compute gradients with **autograd** and discard **autograd**



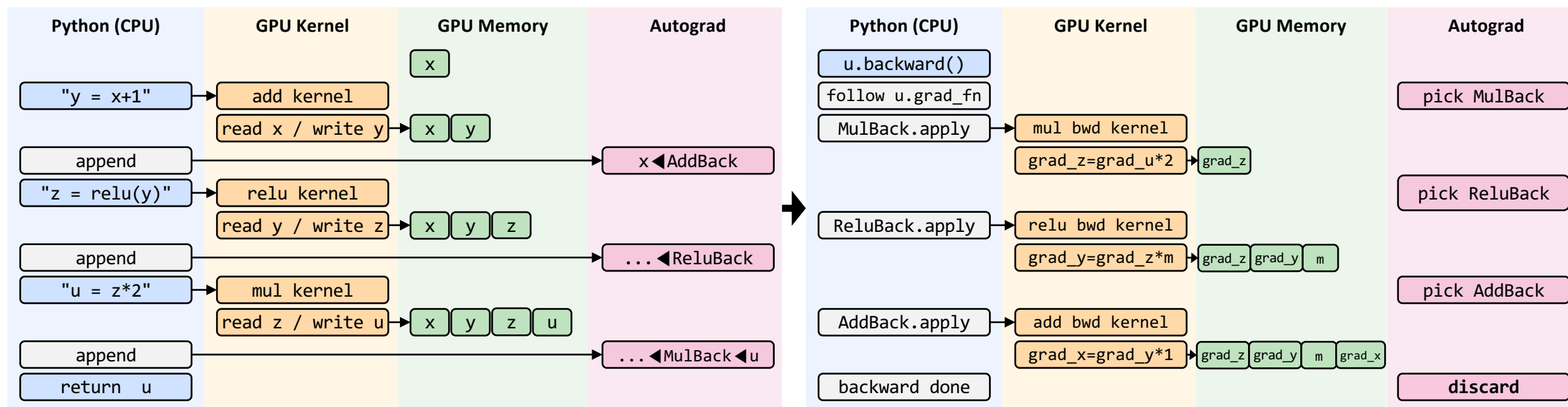
Umm...

## Too Inefficient!



Can we pre-construct “**cache**” or optimize “**plan**” for GPU kernel, memory, and autograd?

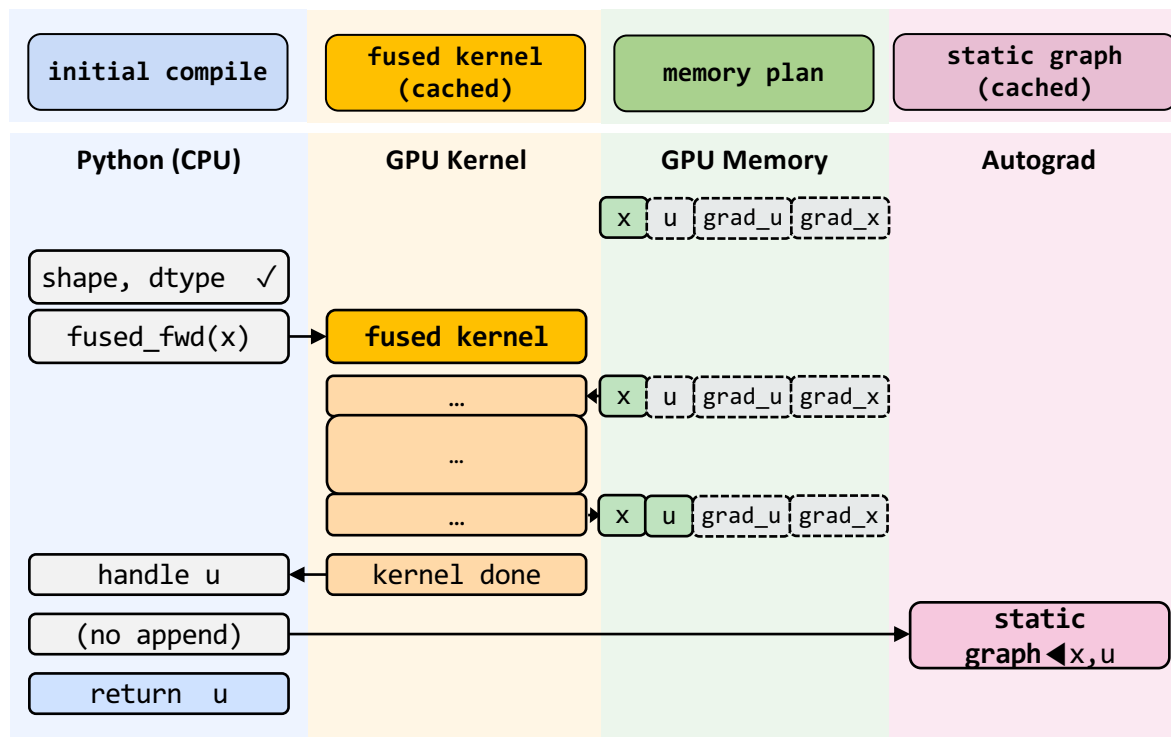
- Next step may use the **identical kernel**, address the **same memory address & size**, and yield the **identical autograd graph**



```
net = torch.compile(net)
```

Torch Compile

# Torch.compile(model)

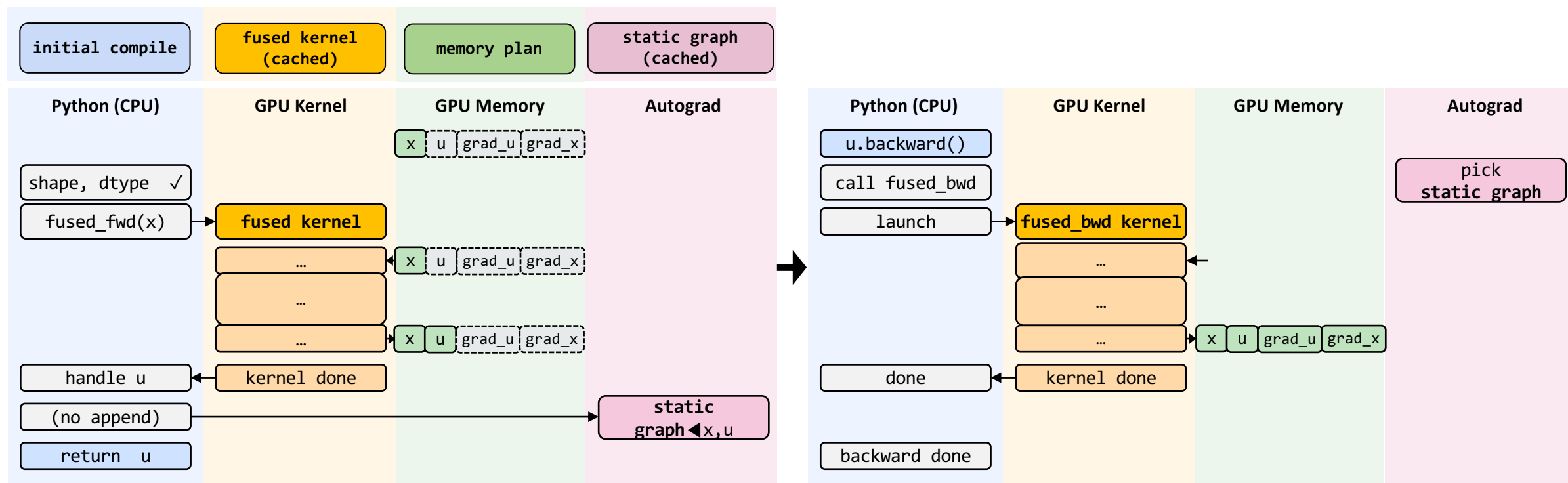


- Construct
  - **Fused kernel:** grouping multiple kernels
  - **Static graph:** a static function to compute gradients
  - **Memory plan:** a compressed&fixed memory allocation at **initial compile time**

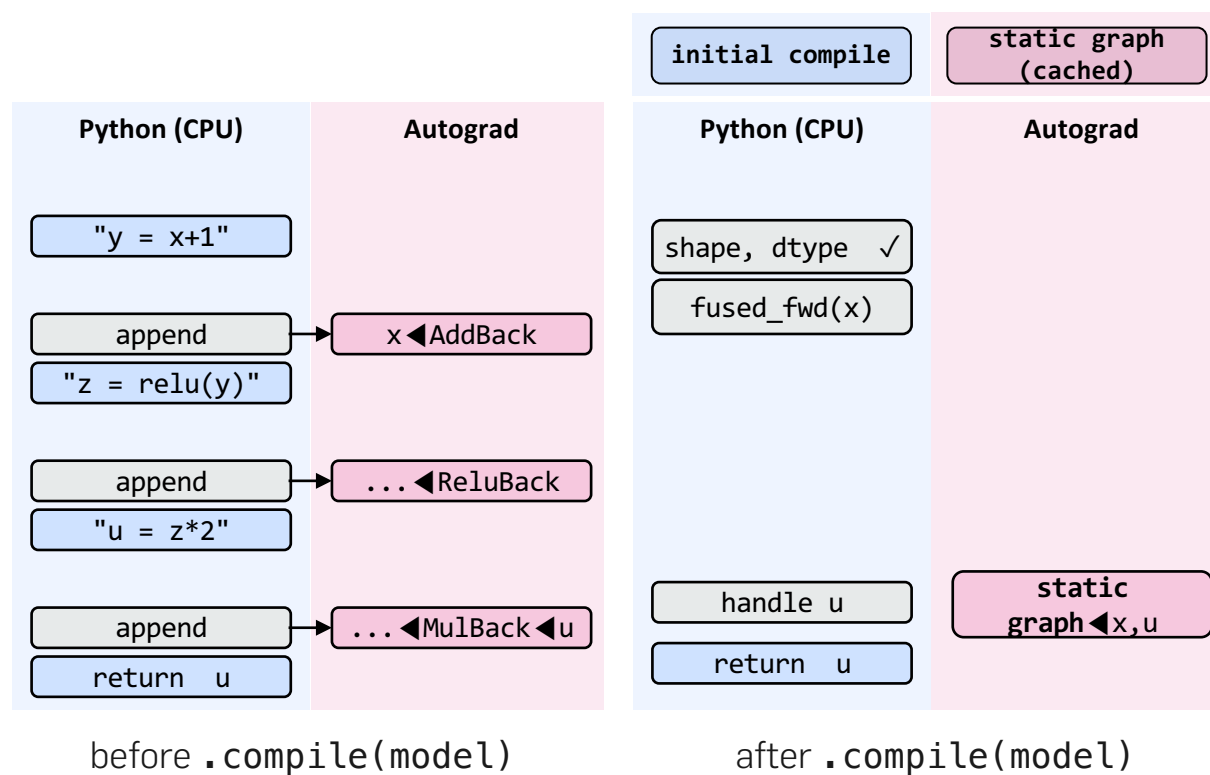
```
net = torch.compile(net)
```

Torch Compile

# Torch.compile(model)



# Construct Static Graph Before Training



- Compile constructs `static graph (cached)` in advance:
  - using pre-built static graph
  - **strong assumption:**  
*same computational graph for inputs*

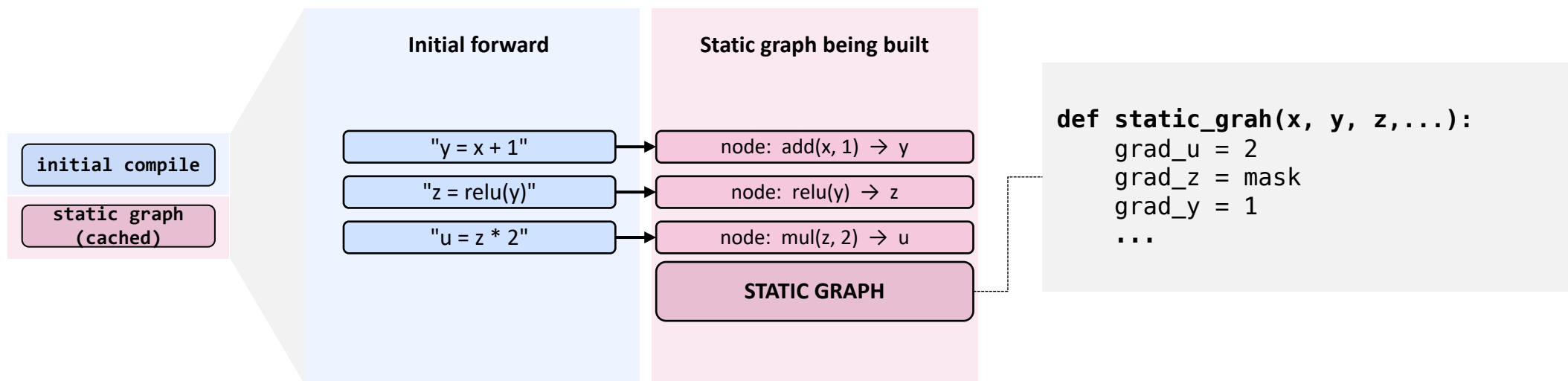
✗

```
if rand() > 0.5:
    x = x * w1
else:
    x = x * w2
```

prohibits branching

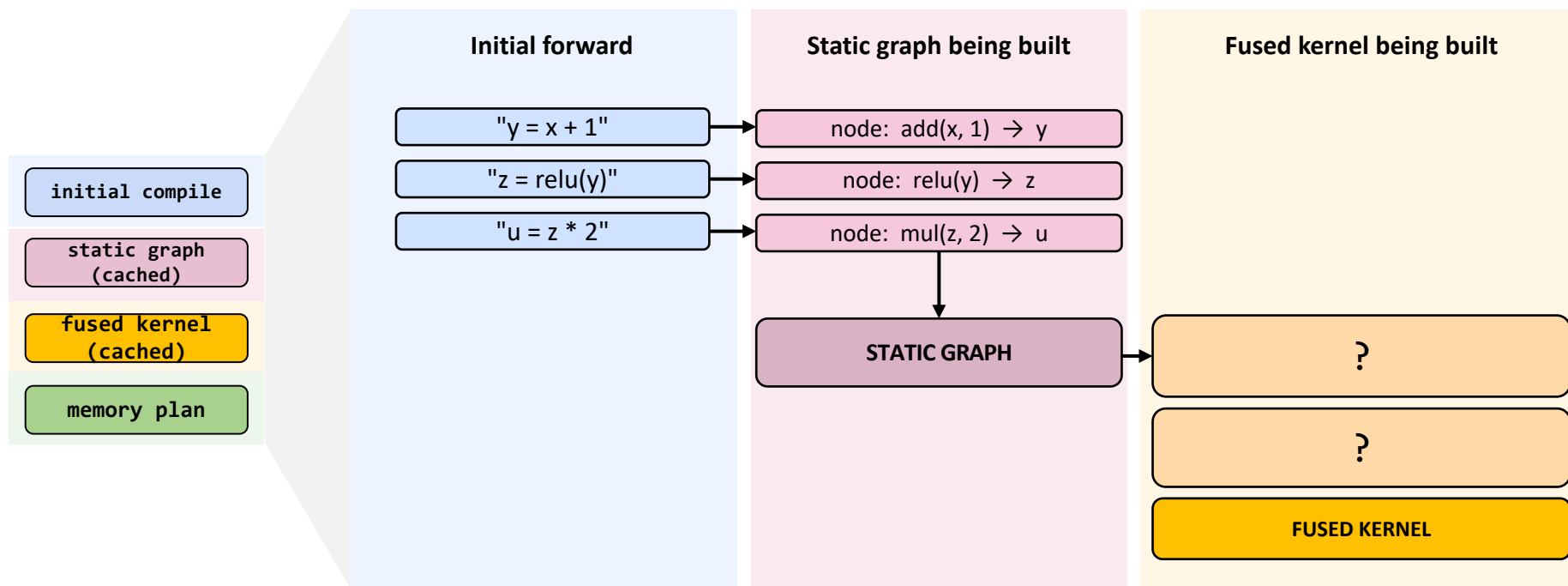
```
mask = rand() > 0.5
x = x*(w1*mask+w2)
```

# Construct Static Graph Before Training



- Trace **forward pass**, record **tensor operations as a graph**
- Constructed **static graph = a function** (operations → body lines, inputs/outputs → args)

# Construct Fused Kernel Before Training





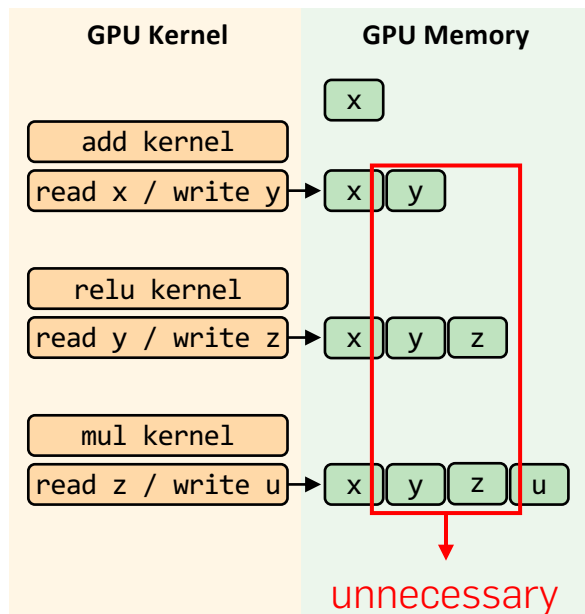
```

y = x+1
z = relu(y)
u = z*2
u.backward()

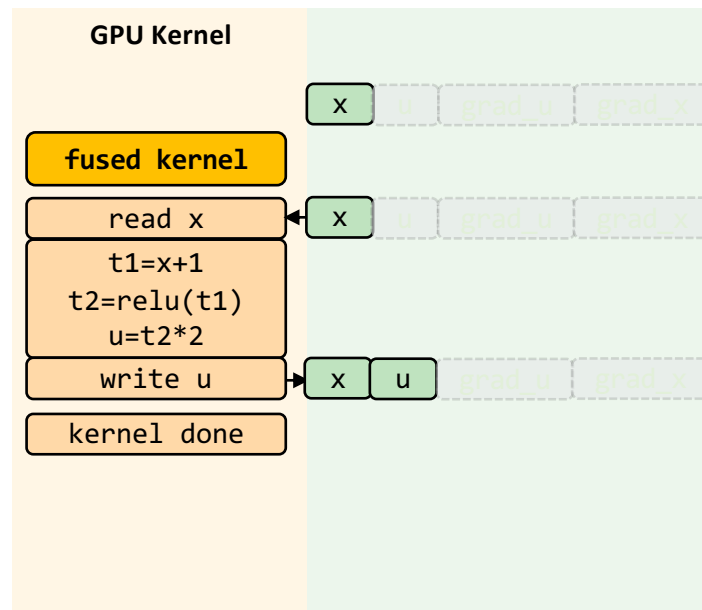
```

# Fused Kernel Construction

before .compile(model)

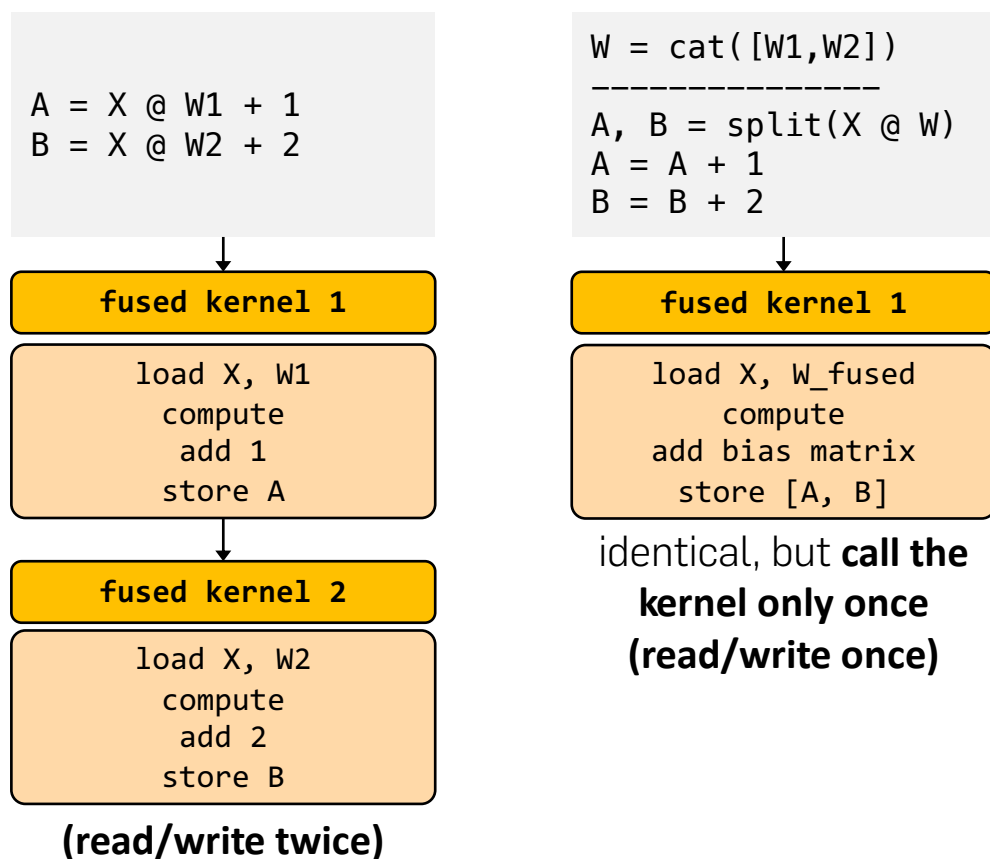


after .compile(model)



- **after .compile(model):**
  - reducing **# kernel calls**
  - reducing **GPU memory reads/writes**
- **fused kernel** :
  - removing **unnecessary writes** (read/write once)
  - grouping **element-wise operations**

# Trick for Kernel Optimization



- **fused kernel** groups:
  - element-wise operations
- **Two matrix multiplications** can not be fused!
  - **MatMul kernel** is assigned **for each**
- **Tip.** Write a code to **minimize # MatMul calls**



Nayoung Kim 오후 1:54

이번에 autoresearch 형태로 model throughput optimization 해봤는데, 생각보다 잘돼서 (약 x6) 실험 내용 공유드립니다 (with code)

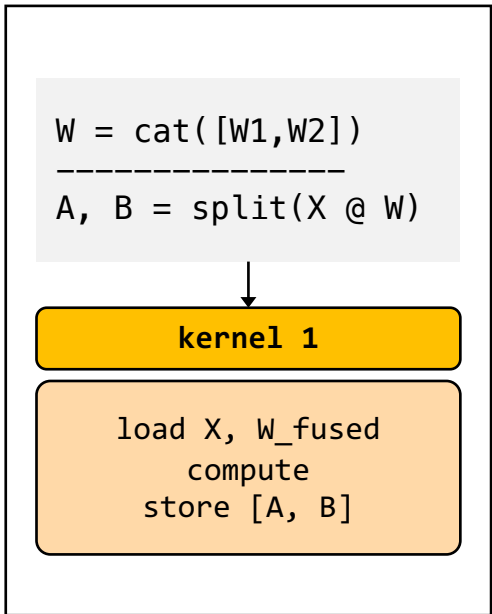
---

Experiment summary: CSD training throughput optimization

Goal

Improve MatterFlow CSD training-step throughput without changing model/training semantics.

shout out to Nayoung Kim



advancing

|   |                                           |                                                                                                                                                                                                                                |          |      |         |
|---|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------|---------|
| d | Reuse scaled targets in loss              | Bitwise exact forward/internal check versus fresh trial_102_reference.pt; old recomputed-target and new reused-target loss paths matched with zero max difference.                                                             | 982.63   | 3.84 | 156,073 |
|   |                                           |                                                                                                                                                                                                                                |          | ↓ x2 |         |
|   | Stack pair-bias projections across layers | Bitwise exact forward/internal check versus fresh trial_103_reference.pt. Isolated projection-gradient check: outputs and parameter grads exact; upstream pair-feature grad max_abs_diff 0.00293 from bf16 accumulation order. | 1,786.94 | 6.98 | 283,824 |

```
W_stacked = cat([W_bias_1, W_bias_2, ..., W_bias_L])
-----
bias_all = einsum("logic", input, W_stacked)
bias_1, ..., bias_L = split(bias_all)
```

# Fused Kernel for Attention vs. Pairformer (Claude)

| PyTorch code                                                                | Eager (~11 kernels)                                                                                                                | Compiled (~3 kernels)                                                                                        | Pairformer code                                                                                                          | Eager (~40-50 kernels)                                                                                                                          | Compiled (~12-16 kernels)                                                                                                                                   |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>x = layer_norm(x) qkv = x @ W_qkv + b Q, K, V = qkv.chunk(3, -1)</pre> | <ul style="list-style-type: none"> <li>layer_norm</li> <li>matmul (x @ W_qkv)</li> <li>bias_add</li> </ul>                         | <p>Kernel 1</p> <p>LayerNorm + QKV GEMM + bias (prologue/epilogue fused)</p>                                 | <pre># 2x Triangle attention # (non-standard: pair bias) pair = pair + tri_attn(pair) pair = pair + tri_attn(pair)</pre> | <ul style="list-style-type: none"> <li>LN, Q, K, V, bias matmuls</li> <li>Q@K.T + bias + softmax</li> <li>P@V + gate + output (x2)</li> </ul>   | <p>Kernels 1-6 (x2 for start/end)</p> <p>LN + QKV + bias GEMMs (fused)</p> <p>Attn-with-pair-bias (manual fuse)</p> <p>Output + gate + residual (fused)</p> |
| <pre>O = F.scaled_dot_product_ attention(Q, K, V)</pre>                     | <ul style="list-style-type: none"> <li>matmul (Q @ K.T)</li> <li>softmax</li> <li>matmul (P @ V)</li> </ul>                        | <p>Kernel 2</p> <p>FlashAttention (pre-built — compile DISPATCHES, no new fusion work)</p>                   | <pre># 2x Triangle mul update # (sigmoid gates + einsum) pair = pair + tri_mul(pair) pair = pair + tri_mul(pair)</pre>   | <ul style="list-style-type: none"> <li>LN + linear_a, _b, _g</li> <li>sigmoid + multiply + einsum</li> <li>LN + linear_o + gate (x2)</li> </ul> | <p>Kernels 7-12 (x2 for out/in)</p> <p>LN + linear_a/b/g + sigmoid (fused)</p> <p>einsum (cuBLAS BMM)</p> <p>LN + linear_o + gate + residual</p>            |
| <pre>out = O @ W_o + b_o x = x + out x = x + mlp(layer_norm(x))</pre>       | <ul style="list-style-type: none"> <li>matmul (output)</li> <li>residual + LN + MLP ops</li> <li>matmul + GELU + matmul</li> </ul> | <p>Kernel 3</p> <p>Output GEMM + residual + LN + MLP (GEMM + GELU + GEMM) → effectively 2-3 mini kernels</p> | <pre># Pair transition pair = pair + transition(pair) # = LN -&gt; Linear -&gt; GELU # -&gt; Linear</pre>                | <ul style="list-style-type: none"> <li>LayerNorm</li> <li>Linear + bias + GELU</li> <li>Linear + bias + residual</li> </ul>                     | <p>Kernels 13-14</p> <p>LN + Linear + GELU (epilogue)</p> <p>Linear + bias + residual (epilogue)</p>                                                        |

- Pairformer significantly benefits from `.compile()` by reducing **# of reads/writes of the pair rep**

## Fused Kernel Construction (Cont.)

fused kernel

```
def fused_kernel(x, out, N, BLOCK: tl.constexpr):  
    pid = tl.program_id(0)  
    offs = pid * BLOCK + tl.arange(0, BLOCK)  
    ...
```

low-level fused kernel optimization (triton) uses  
**BLOCK (depending on dtype/shape)**

- **Strong assumption** in a **fused kernel**:  
**Dtype and shape** of every tensor **remain unchanged**
- The fused kernel is **optimized with tensor dtype/shape**

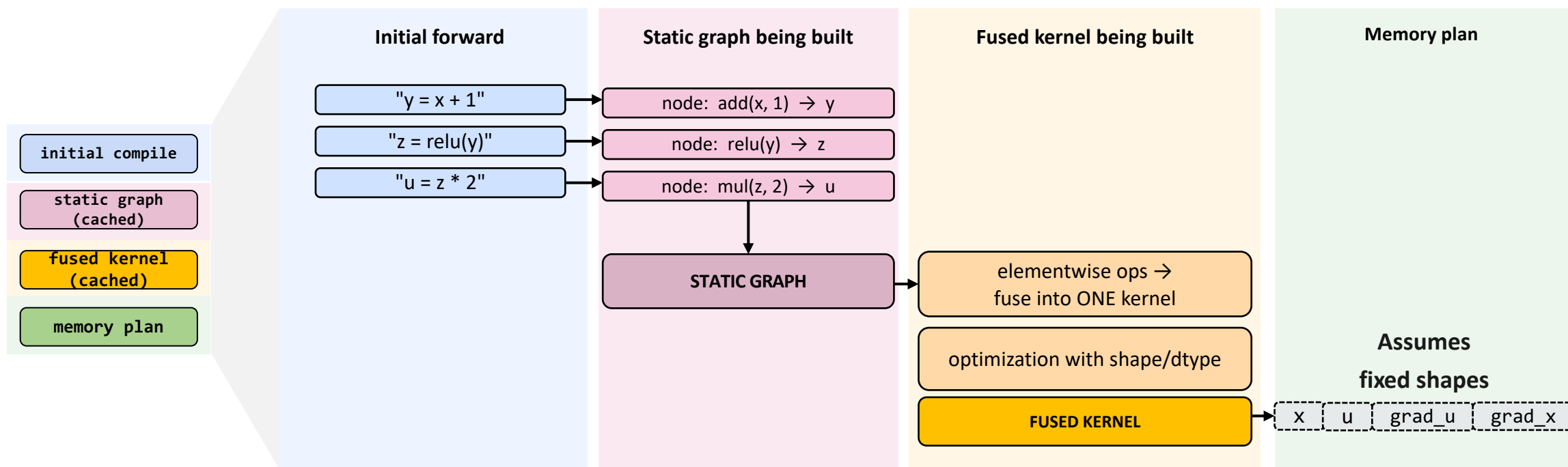


```
z_mask = z[mask]  
x[mask] = z_mask
```

```
x[mask]=z [mask]
```

Matters when the **mask**  
**shape differs across calls**

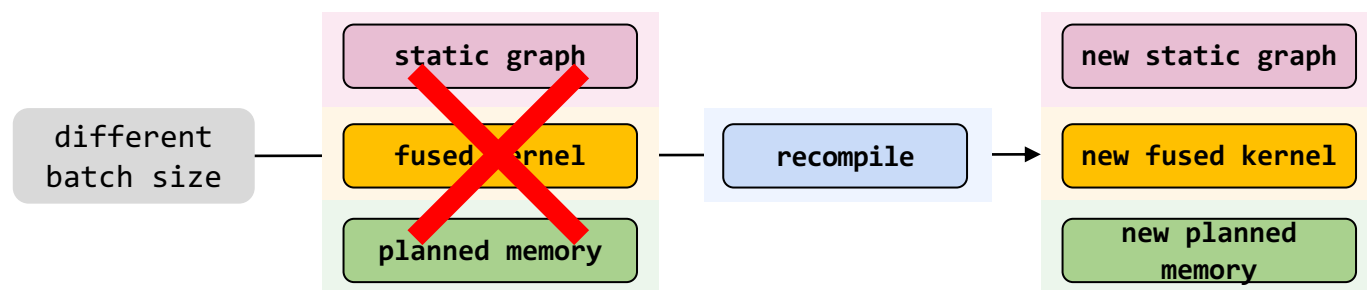
# Construct Fused Kernel Before Training



- **Planned memory** allocates a **fixed address** and **size** for each tensor

Fallback

# Recompile



- Triggered when **shape change**
- Redoes the **full compile pipeline**
- Recompile requires few seconds but **shifts memory layout**

- Actually, **sparse recompilation does not matter** much in **single-node training**, but even **one recompilation can matter** in **multi-node training** (details in later)

## Wrapup

# Torch.compile()



```
if rand() > 0.5:  
    x = x * w1  
else:  
    x = x * w2
```

```
mask = rand() > 0.5  
x = x*(w1*mask+w2)
```

graph consistency



```
z_mask = z[mask]  
x[mask] = z_mask
```

```
x[mask]=z[mask]
```

shape consistency



```
A = X @ W1 + 1  
B = X @ W2 + 2
```

```
W = cat([W1,W2])  
---  
A, B = split(X @ W)  
A = A + 1  
B = B + 2
```

# kernel calls

- Always check **graph and shape consistency**
- Think about **how fused kernel are constructed** and write a code that **reduces # MatMul kernel calls**



## Overview

# Computation Time

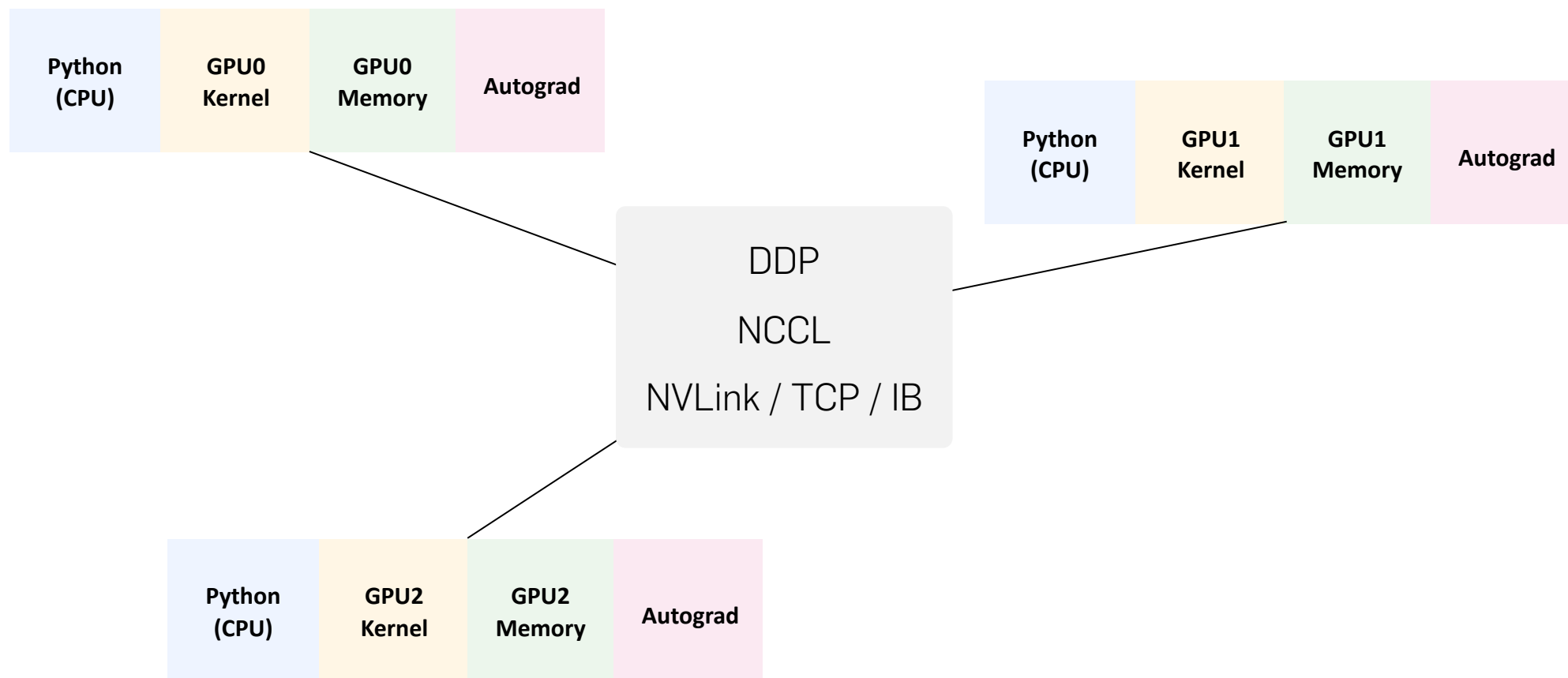


How can we optimize these?



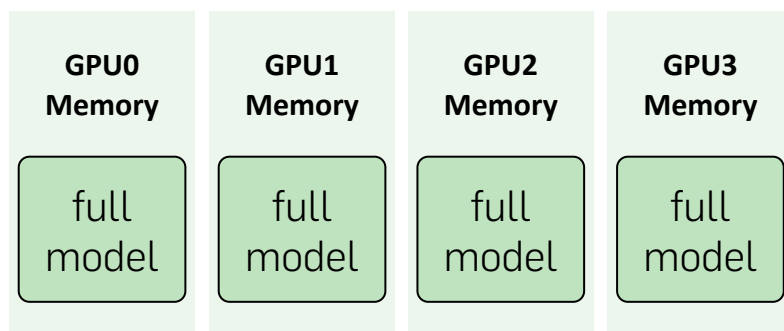
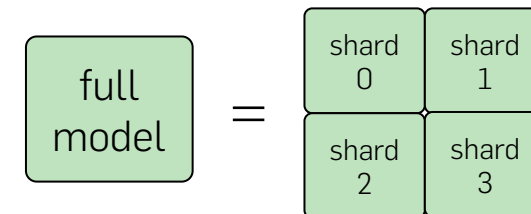
- **Forward and backward computation** time matters in general
- **Communication** overhead matters in **multi-GPU training**
- **Fetch to GPU** time matters when **probing high-dimensional representations** (e.g., pair reps)

# Inter-GPU Communication

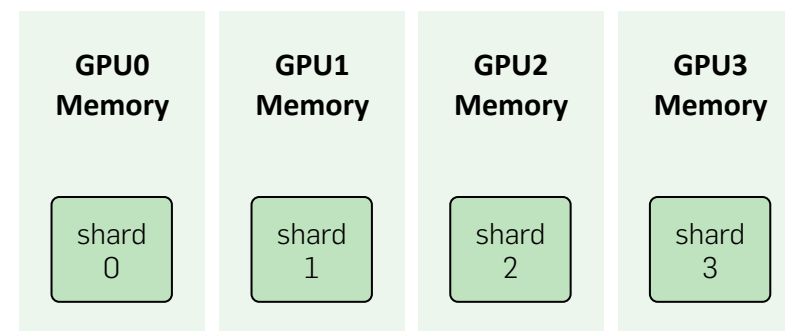


## Multi-GPU training

# DDP vs. FSDP



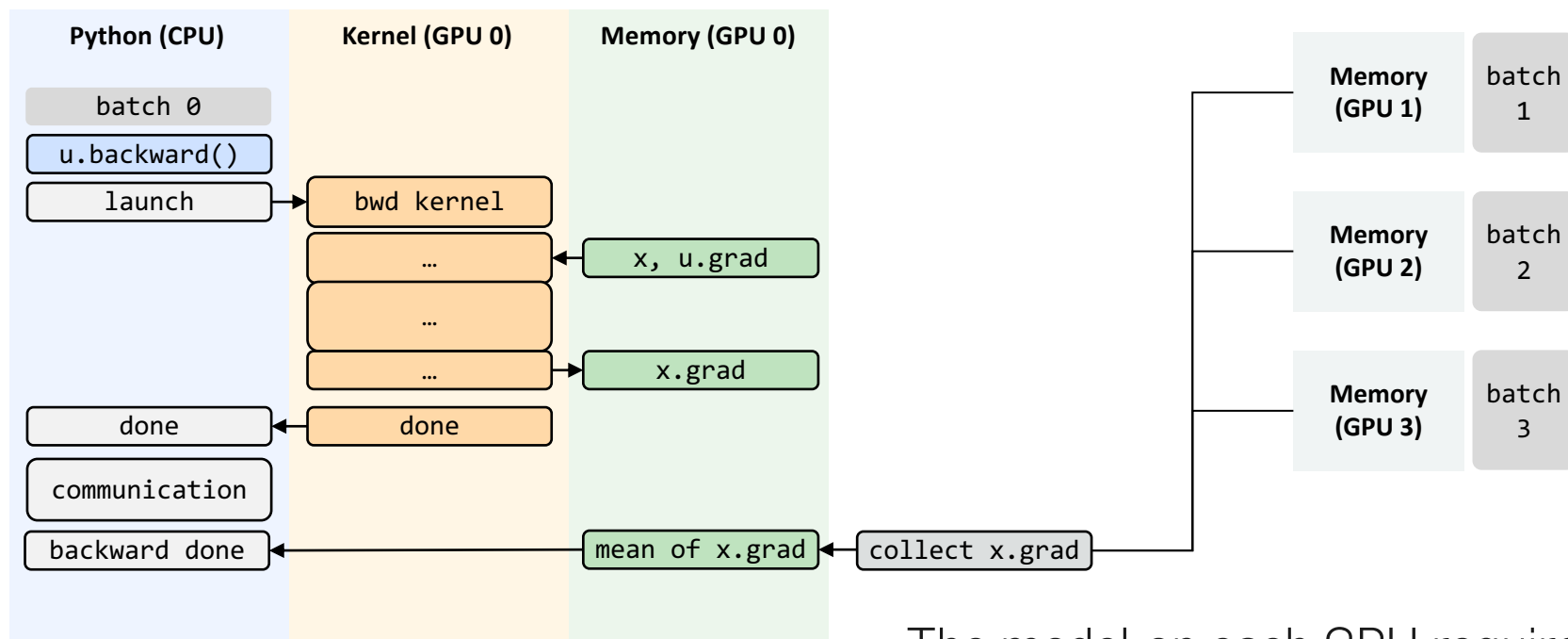
**DDP (DistributedDataParallel)**



**FSDP (FullySharedDataParallel)**

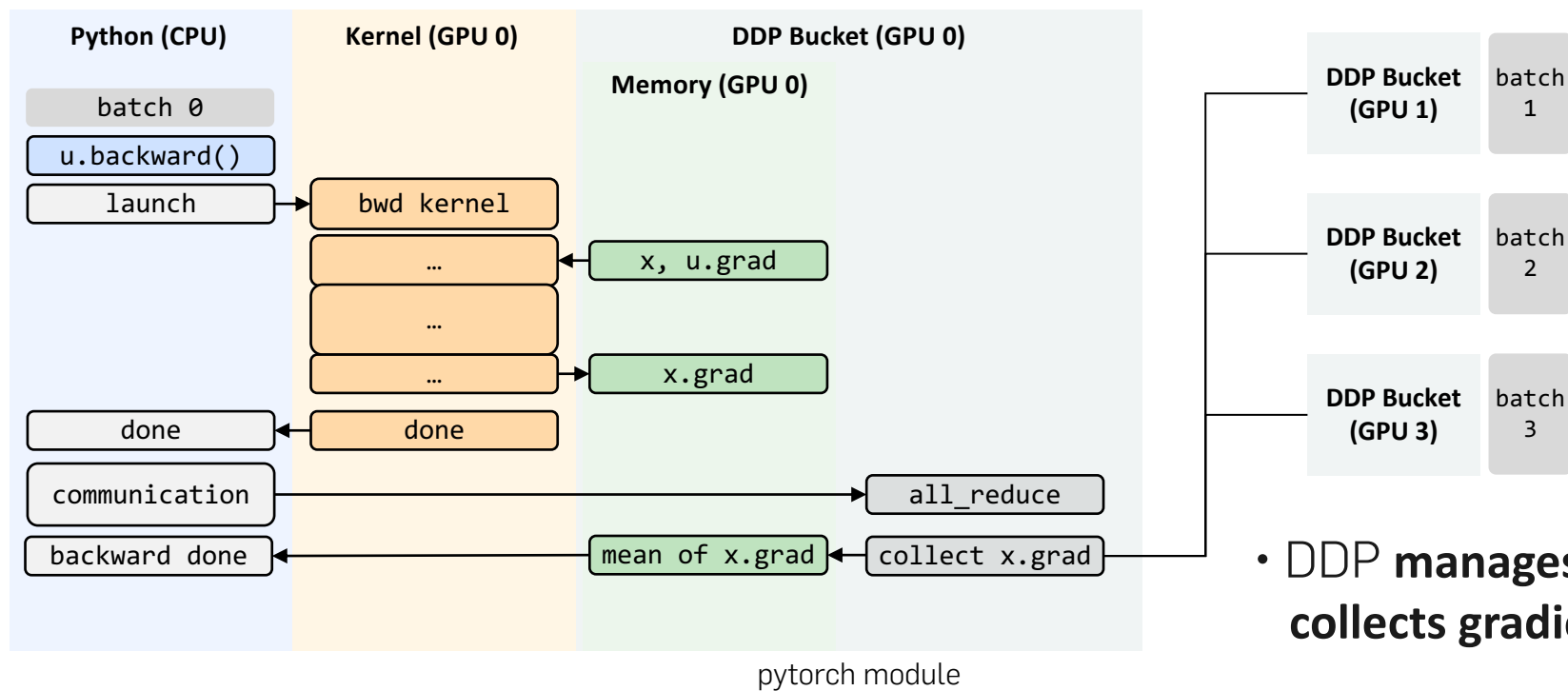
- **DDP** allocates the **full model on each GPU**, while **FSDP** allocates only **part of the model (shard)**
  - **DDP** is designed for handling **large global batch size**, with a small batch on each GPU
  - **FSDP** is designed for training **huge models** (e.g., 70B models)

## Inter-GPU Communication: DDP



- The model on each GPU requires **gradient from all GPUs**
- How to **communicate across GPUs**?

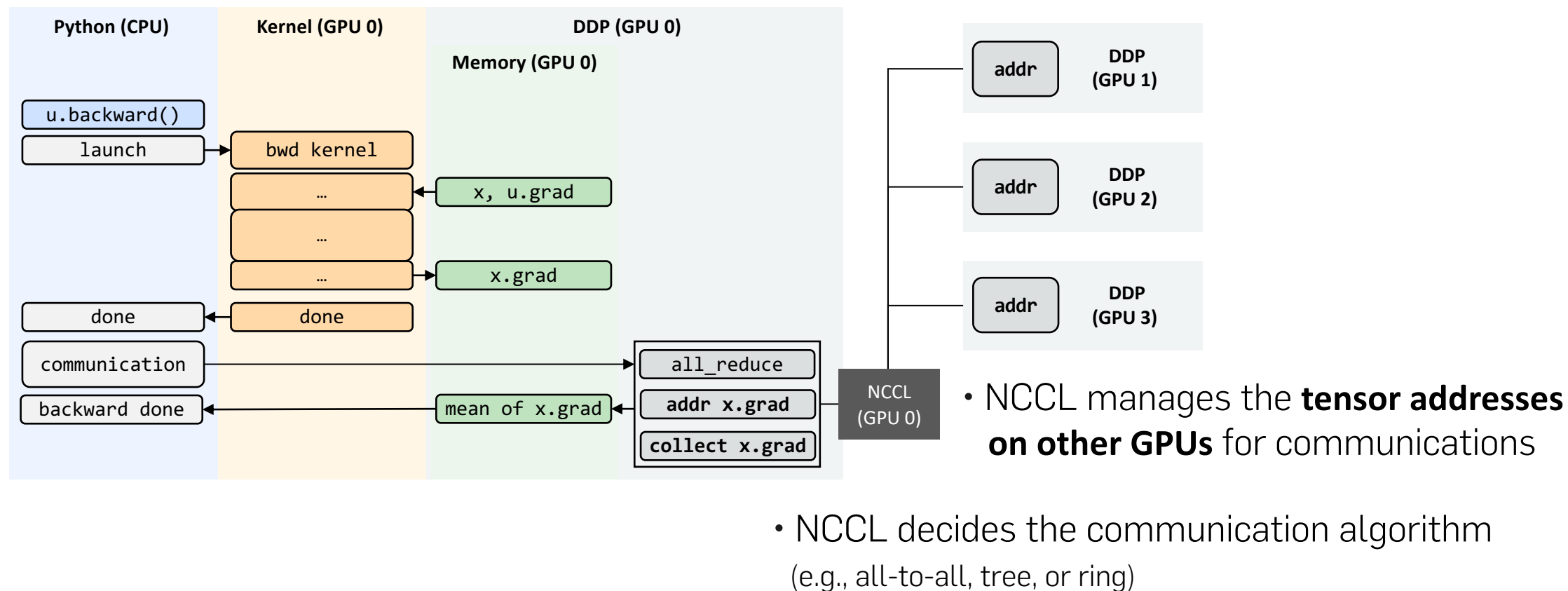
# Inter-GPU Communication: DDP



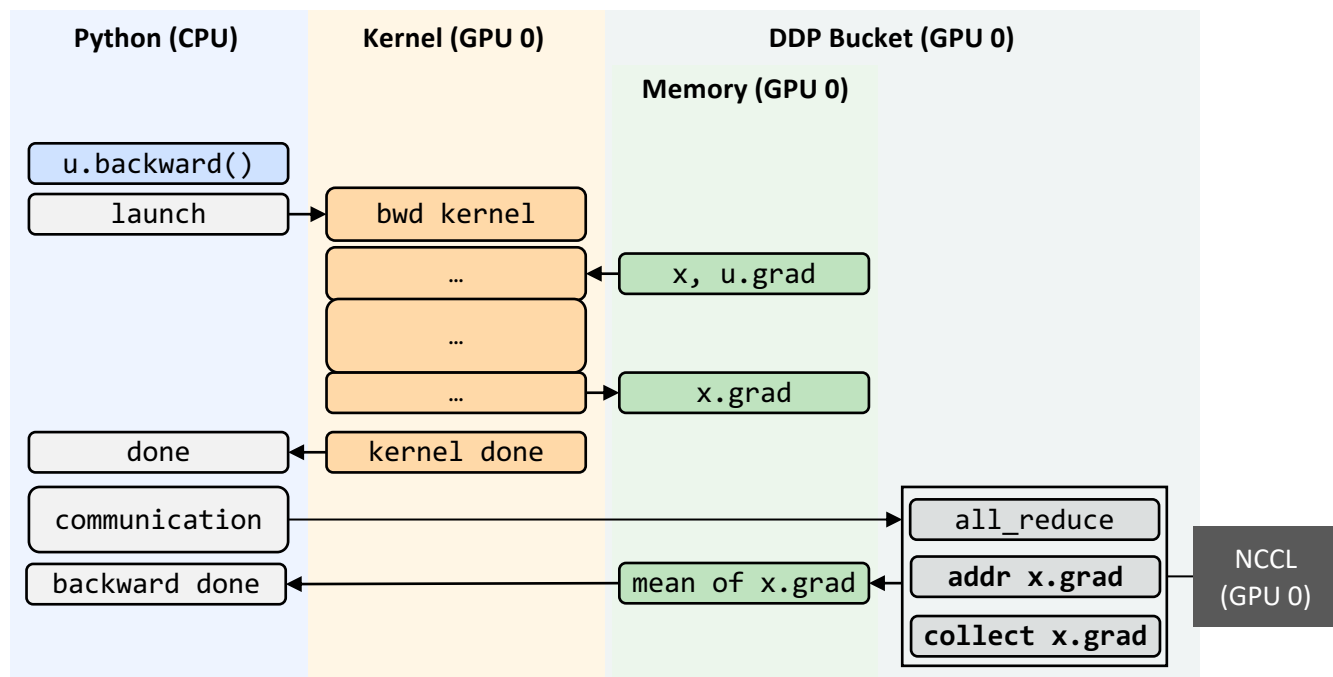
- DDP manages GPU memory and collects gradients from other GPUs

## NVIDIA Collective Communication Library (NCCL)

# Inter-GPU Communication: NCCL



# Inter-GPU Communication: NCCL



pre-establishes **peer connections** between **intra-node** and **inter-node GPUs**

```
for peer in peers:
    if transport[peer] == intra-node:
        peer_ptr[peer] = cudaMemHandle(...)
    elif transport[peer] == inter-node:
        ...
```

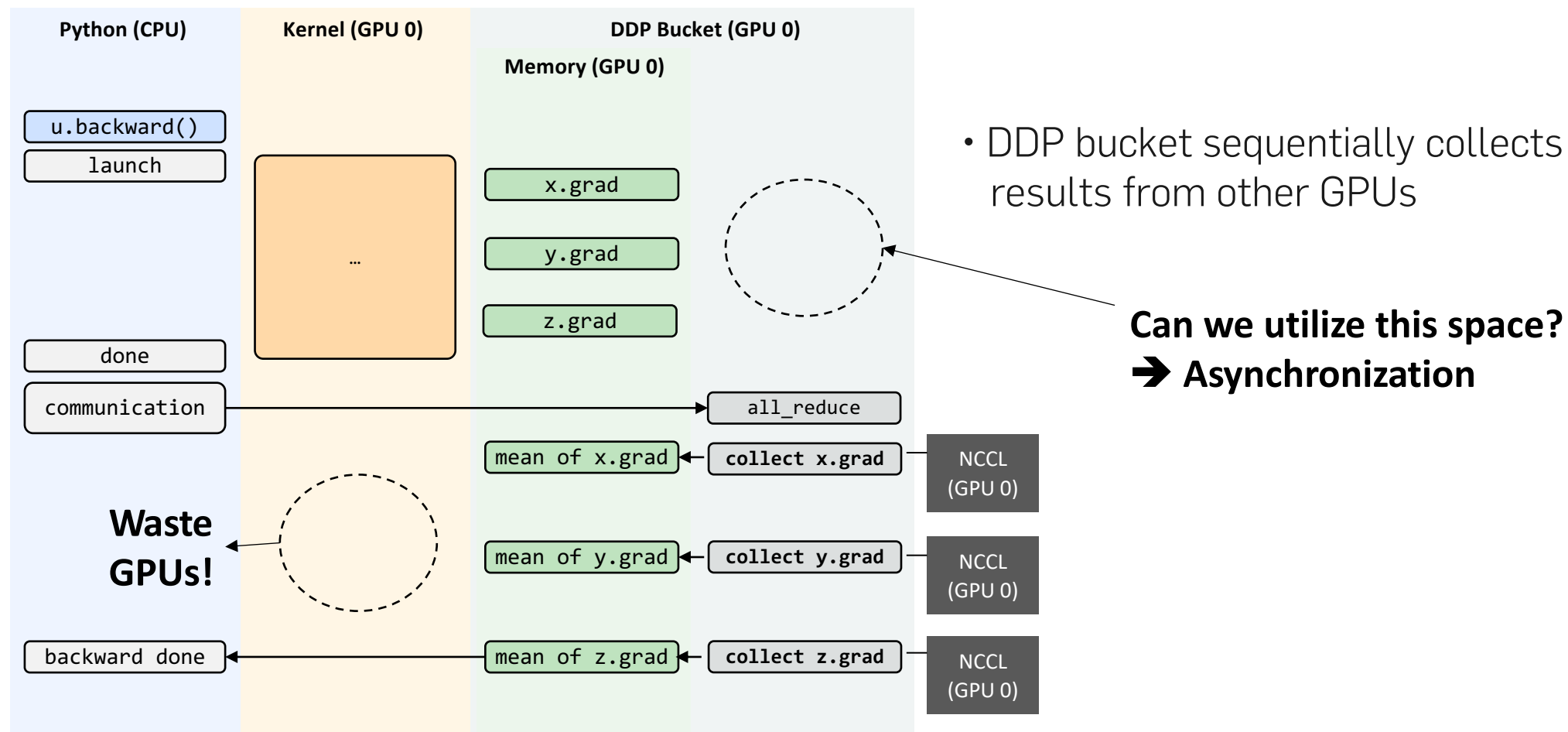
```
def all_reduce(tensor):
    for peer in peers:
        if intra-node:
            my_chunk += peer_ptr[peer][offset]
        elif inter-node:
            ...
```

transfers data via peer connections

Intra-node vs. Inter-node? Details in later!  
CUDA\_VISIBLE\_DEVICES → setup `cudaMemHandle`

## Realistic Example

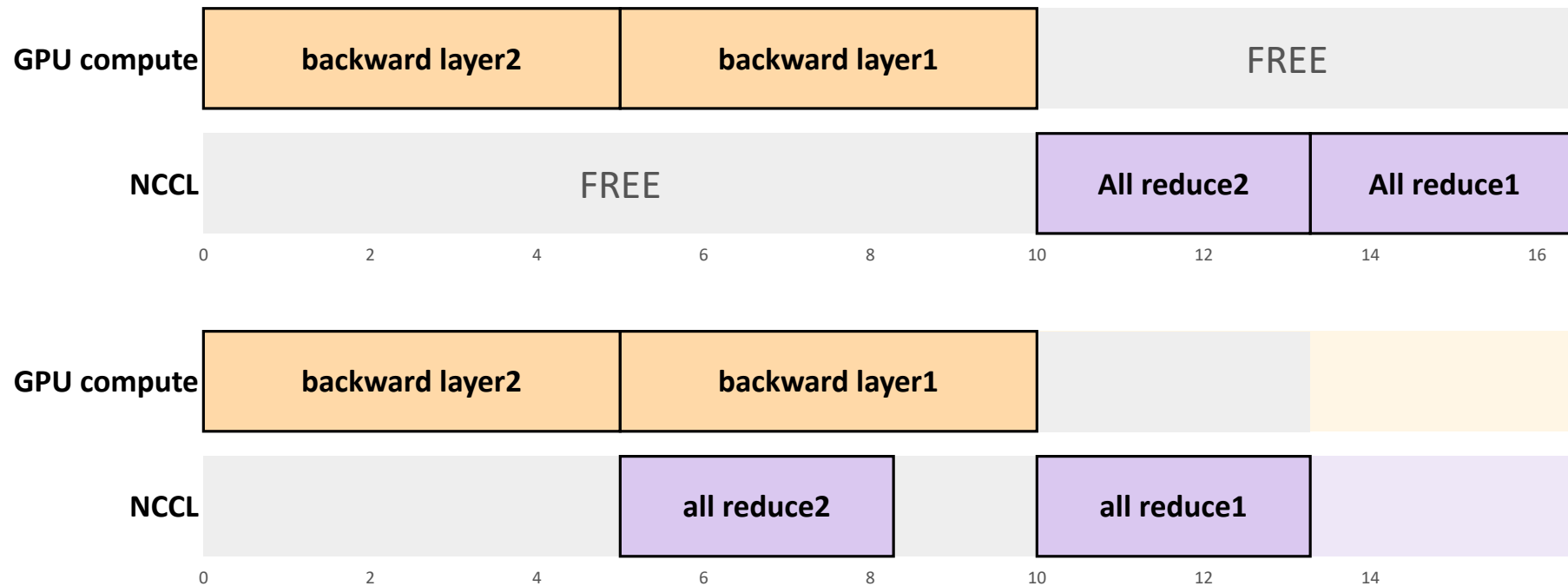
# DDP over Multiple Parameters





Asynchronization between GPU vs. NCCL?

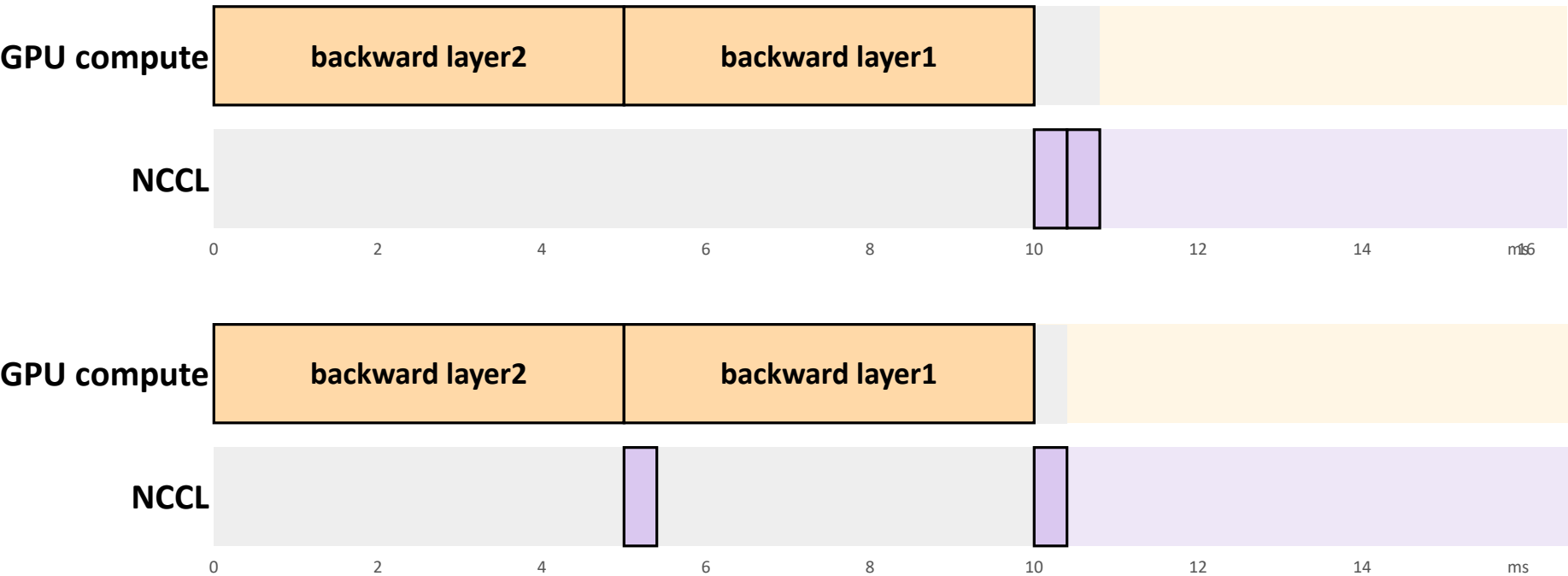
# Inter-GPU Communication: Asynchronization



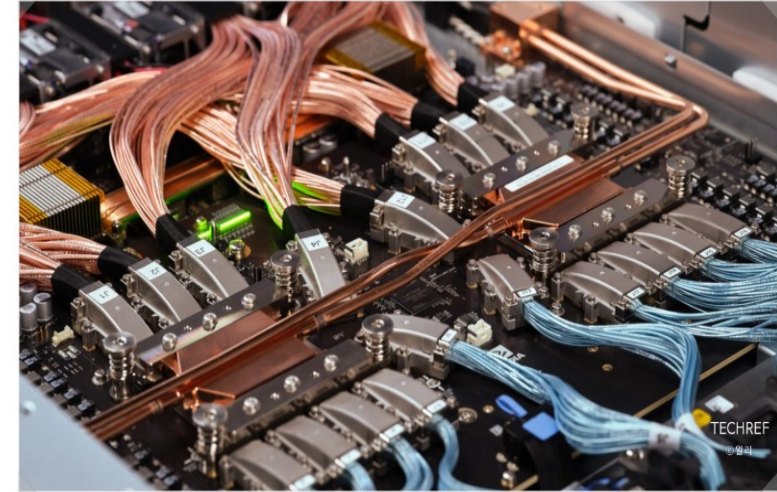
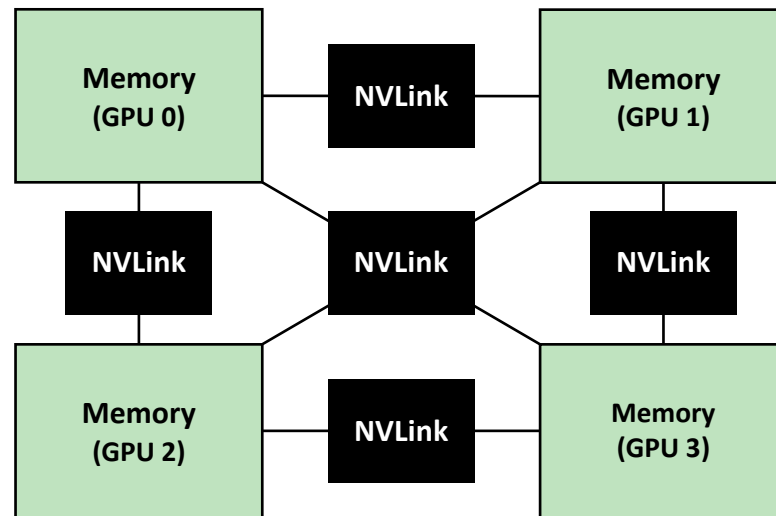
- Fun fact: Vanilla DDP w/o compile uses **asynchronous NCCL** at default

Asynchronization between GPU vs. NCCL?

# Is Asynchronization Useful? Partially, No (in single-node GPUs)



# Hardware Support: NVLink

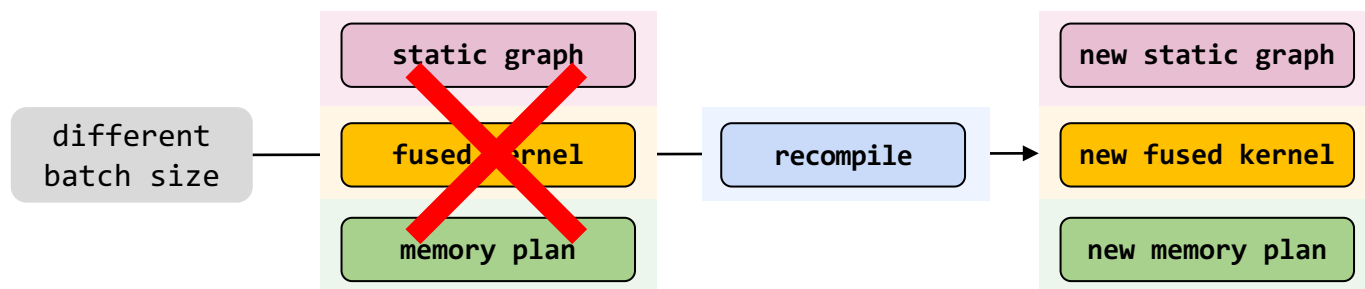


NVLink cable (300GB/s~2TB/s)

- NVLink is the **hardware memory bus** between **GPUs in the same node**
- **NVLink** reads/writes peer memory **directly**

## Fallback

# Recompile is Minor Issue

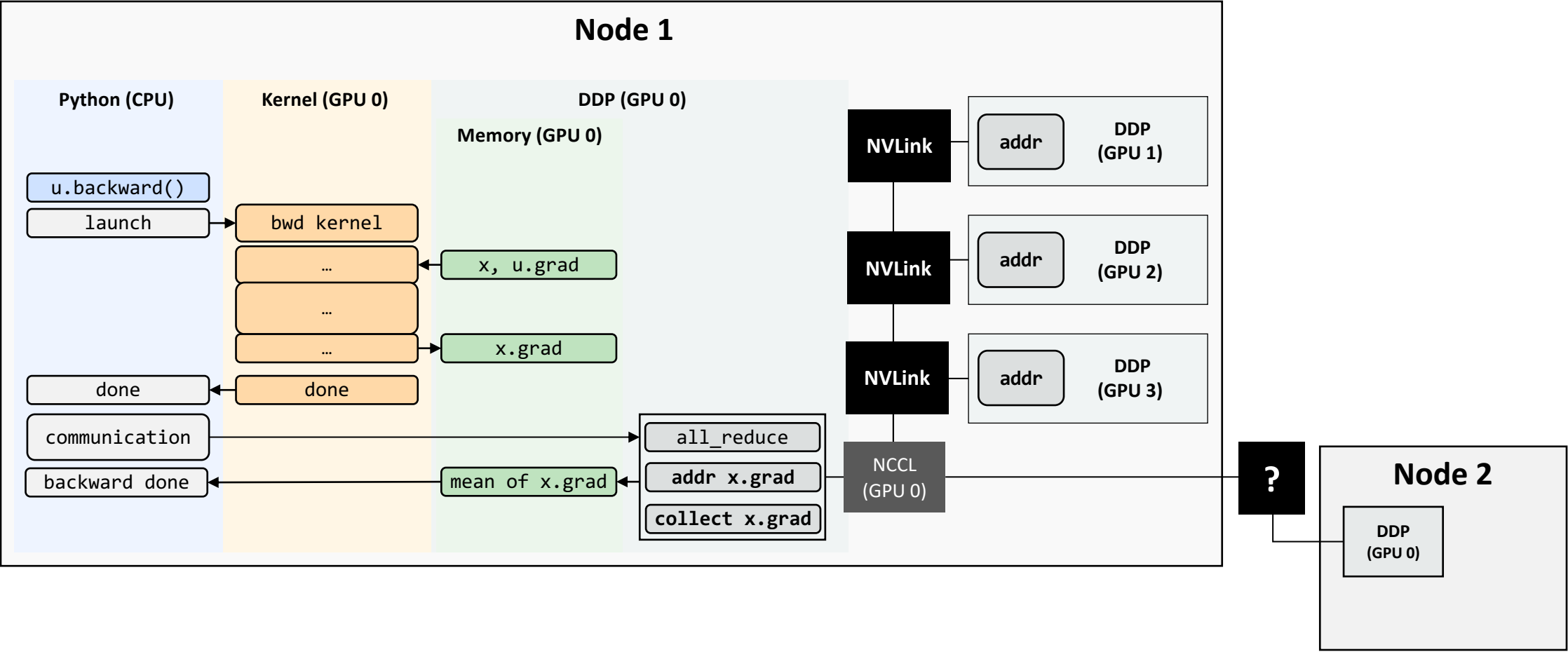


- Changing the **memory address offset** does not matter for **NVLink**-based communication

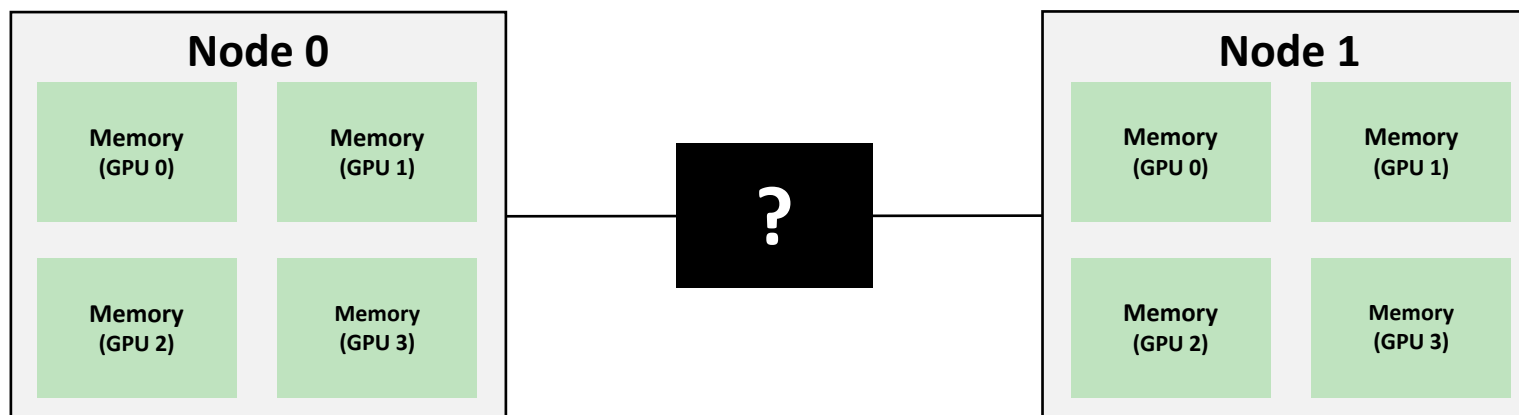
```
for peer in peers:
    if transport[peer] == intra-node:
        peer_ptr[peer] = cudaMemHandle(...)
    elif transport[peer] == inter-node:
        peer_info[peer] = (peer_addr, peer_rkey)
        ibv_reg_mr(staging, ...)

def all_reduce(tensor):
    for peer in peers:
        if intra-node:
            my_chunk += peer_ptr[peer][offset]
        elif inter-node:
            IB.submit(READ,
src=peer_info[peer],dst=local_buf)
            my_chunk += local_buf
```

# Inter-node Communication

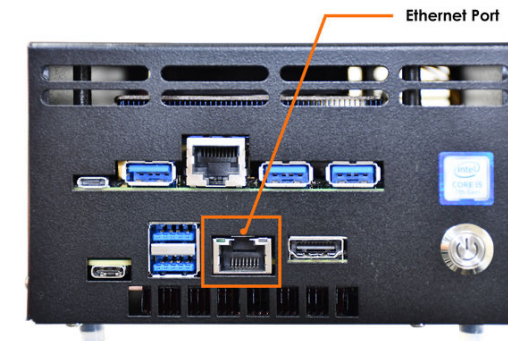


## Inter-node Communication

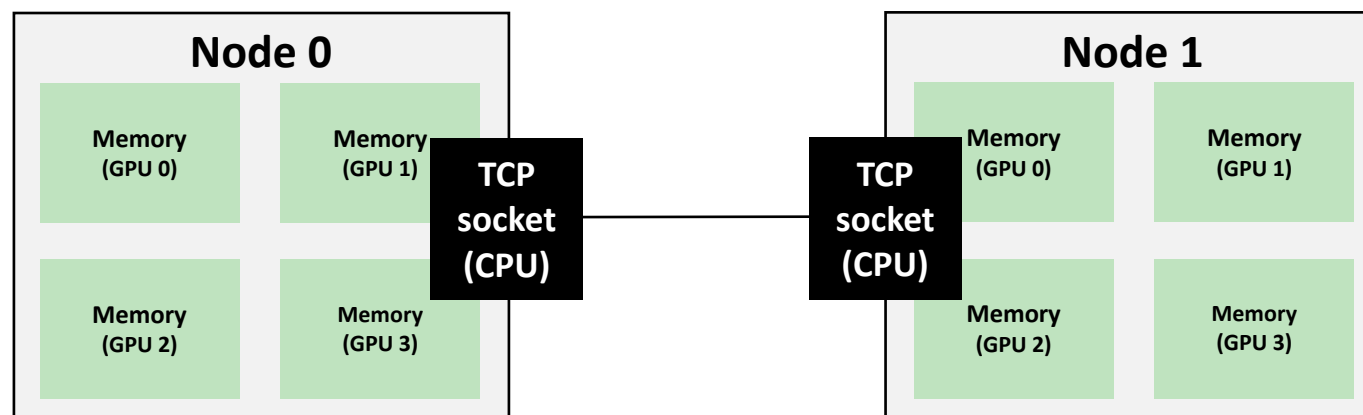


- Two nodes are physically far from each other
- (Optional detail) The topology of multi-node multi-GPUS is **NVIDIA fabric**
- (Optional detail) **NVIDIA fabric manager** finds the **fastest route between two GPUs**

## Inter-Node Communication: Ethernet



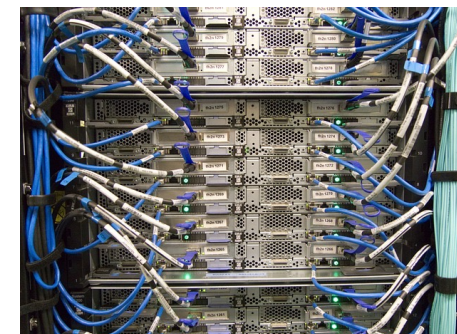
Ethernet (3GB/s~10GB/s)



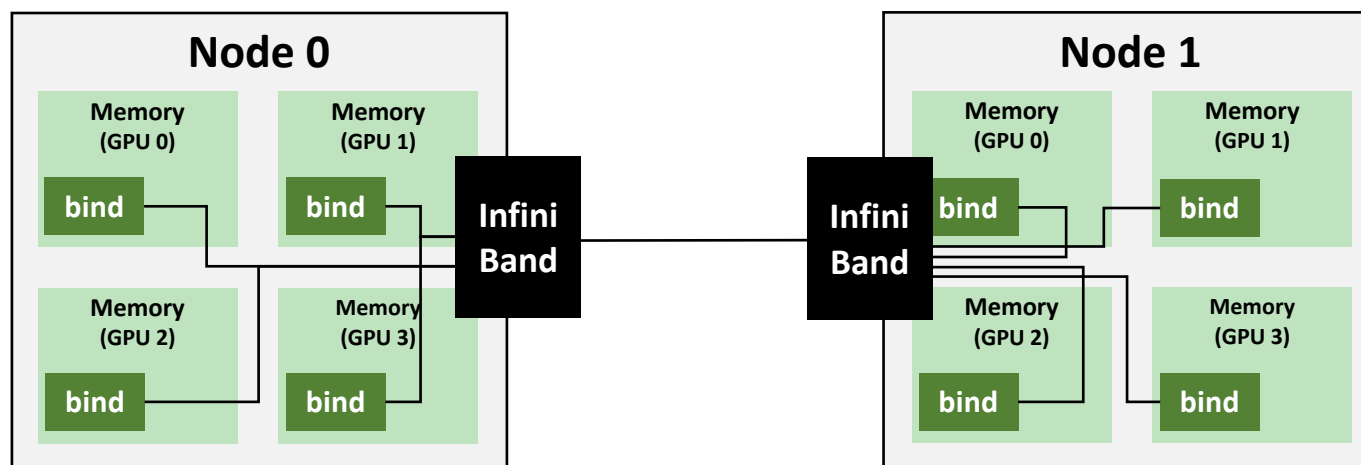
```
# Init
sock = socket.connect(peer_ip, port)
# Send
host_buf = cuda_alloc_pinned(size)
cudaMemcpy(host_buf, gpu_buf, HOST)
sock.send(host_buf)
```

- Ethernet has no direct view of GPU memory
- Path : GPU → host RAM → buffer → Ethernet
- CPU runs the TCP stack on every packet ➔ **too slow!**

## Inter-Node Communication: InfiniBand (IB)



InfiniBand (50GB/s)

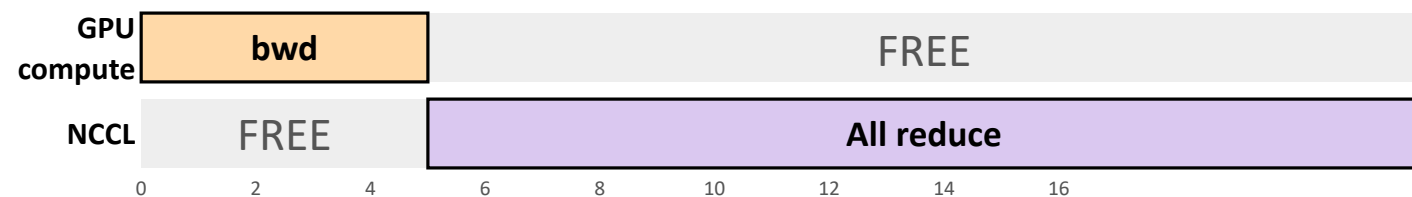


```
# Init
ibv_reg_mr(my_buf, REMOTE_RW)
# Directly write
IB.write(
    src = my_buf,
    dst_addr = peer.addr,
    dst_rkey = peer.rkey)
```

- InfiniBand can **directly read/write to specific memory regions** ( `bind` , **initialized only once** )
- Path : GPU → InfiniBand → GPU (w/o CPU) ➔ **fast!**



## Inter-Node Communication: InfiniBand (IB)

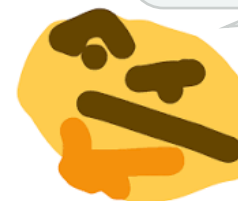


Ethernet (3GB/s~10GB/s)



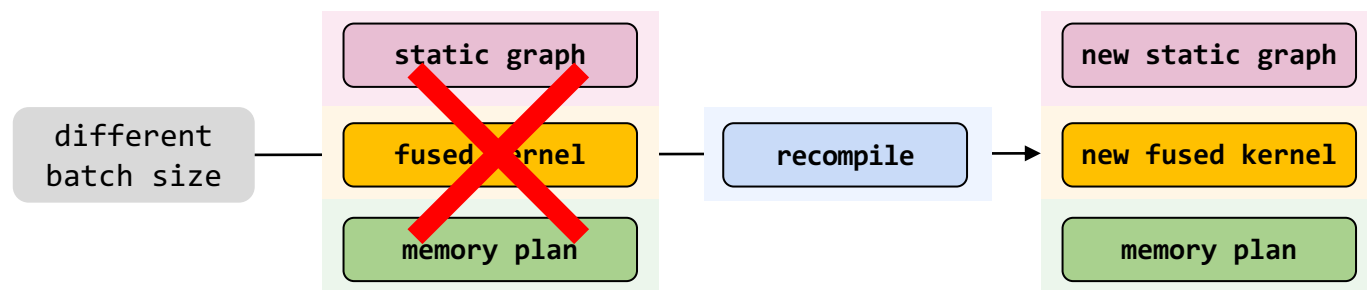
InfiniBand (50GB/s)

Is everything okay with this?



Fallback

## Need to Revisit This (1)



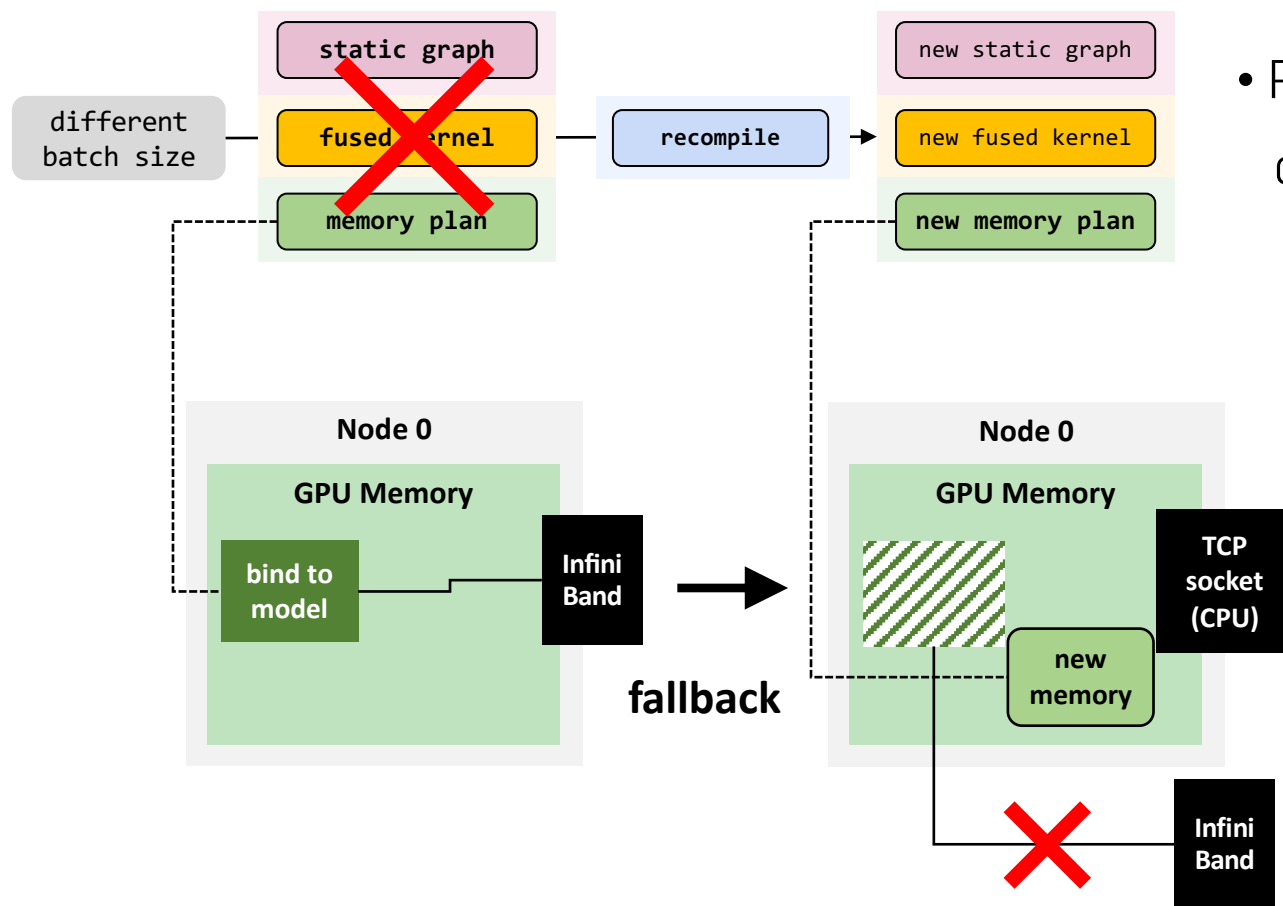
Is this no matter for **InfiniBand**?



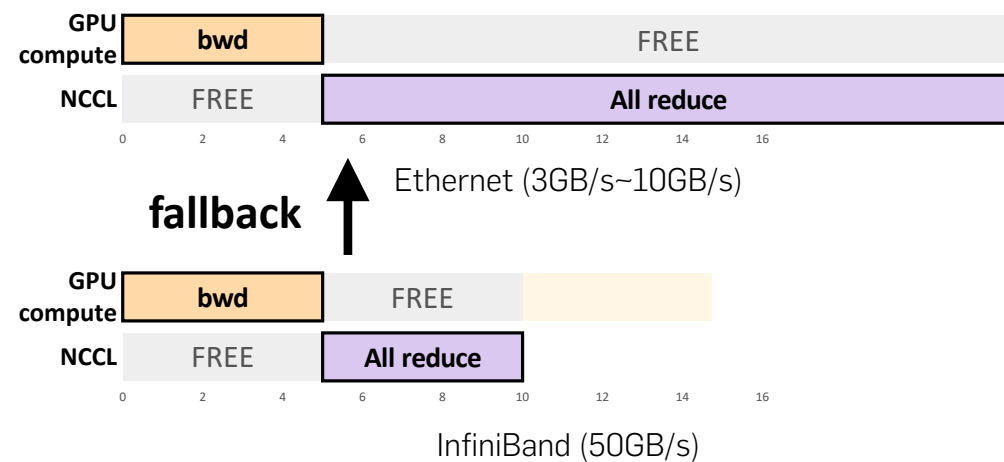
- Changing the **memory address offset** does not matter for **NVLink**-based communication

## Failure Example

# Recompile is Major Issue



- Recompile makes a fallback (InfiniBand → Ethernet) due to to **new memory plan**



## Failure Example

# Recompile is Major Issue

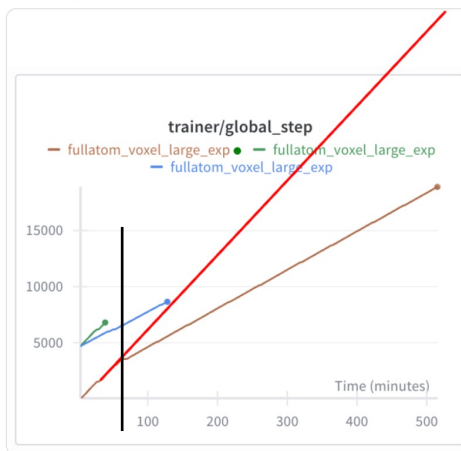
- At the **end of an epoch**, the **batch size changes** in the old code...
- This **causes recompilation** and makes **InfiniBand fall back to Ethernet**



Hyosoon Jang 오후 1:43

아까 말씀드린 이슈에 대하여 원래는 빨간 라인 수준으로 찍혀야합니다 (50분에 1epoch. 지금은 100분에 1epoch) (편집됨)

image.png ▼



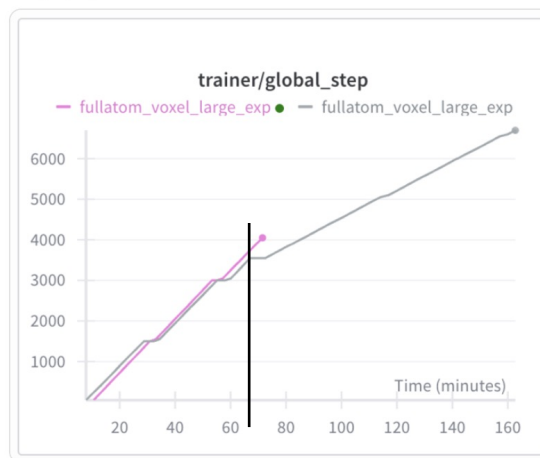
end of epoch



Hyosoon Jang 1월 13일 오후 6:51

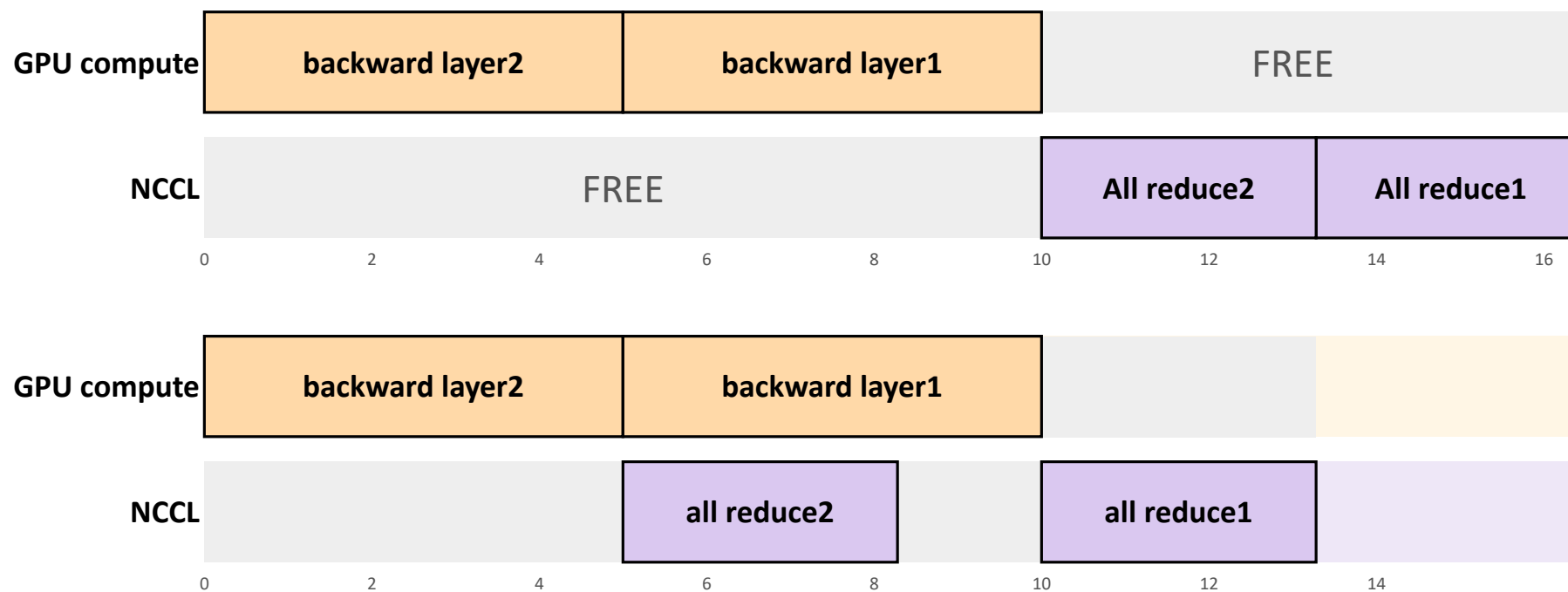
해결했습니다 ㅠㅠ (이제 50분에 1epoch) (편집됨)

image.png ▼



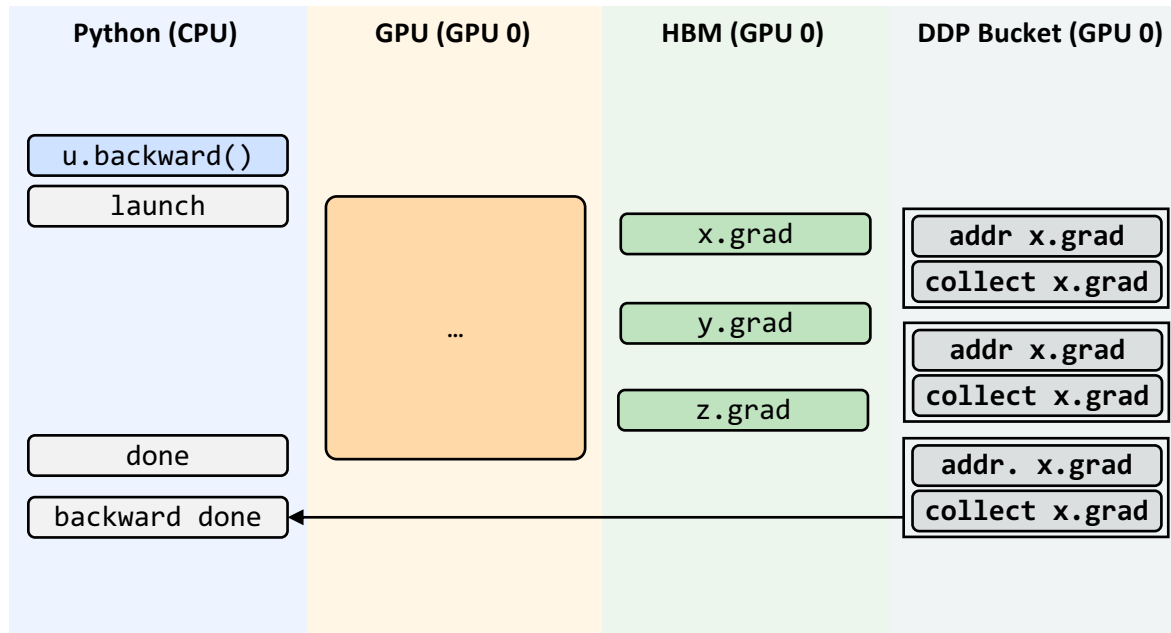
end of epoch

## Need to Revisit This... (2)



- Fun fact: Vanilla DDP w/o compile uses **asynchronous NCCL** at default

# Asynchronization of NCCL in .compile()



Compiled Autograd: Capturing a larger backward graph for `torch.compile`

Created On: Oct 09, 2024 | Last Updated: Mar 31, 2026 | Last Verified: Oct 09, 2024

```
torch._dynamo.config.compiled_autograd = True
```

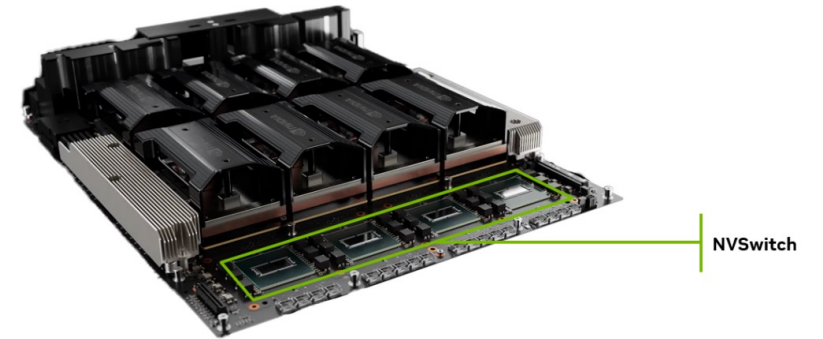
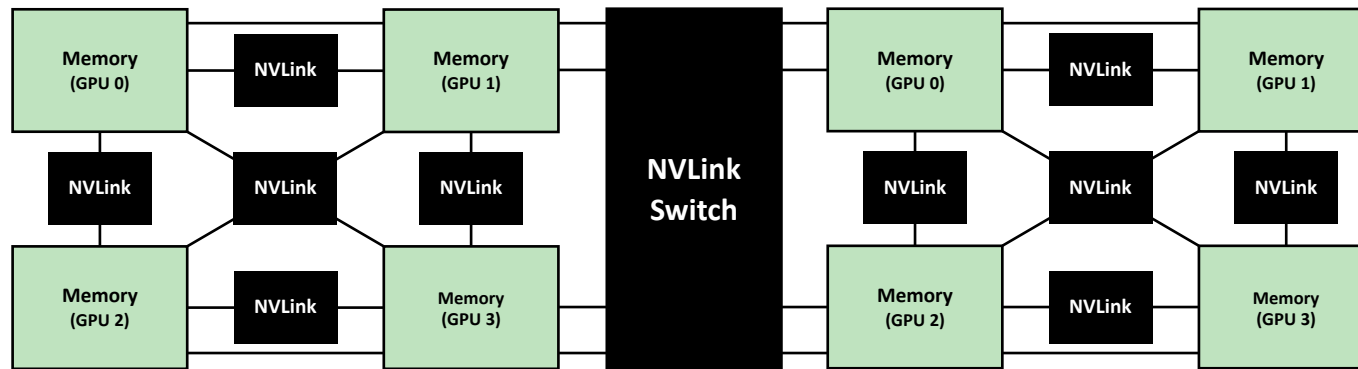
- Compile **entire backward pass**, including **all reduce**
- **Static graph** is **necessary**

## Wrapup

# Inter-node Training

- Compile + IB  
(risk: recompile)
- **Compile + IB + asynchronous NCCL**  
**(risk: recompile + fallback due to version issues)**
- (Easy) No compile (+ asynchronous NCCL) + IB  
(no risks)

# Inter-Node Communication: NVLink Switch



NVLink Switch (~2TB/s)  
**(expensive)**

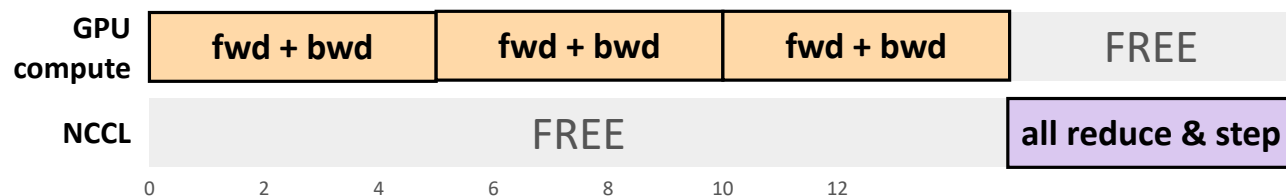


Useful Tip?

## Accumulation of Gradient



No Accum. Grad



3 x Batch Size w/ Accum. Grad

- Increase the **batch size & learning rate** with **accum grad** (communication after accum grad)
- Increase the relative GPU time with respect to NCCL time

## Wrapup

# Intra vs. Inter Node Communication

- **Intra-node multi-gpu training:**
  - Tolerance to change in memory (allowing recompile) thanks to **NVLink**
  - Dynamic graphs&memory changes do not cause a significant throughput reduction
- **Inter-node multi-gpu training:**
  - **Best: Compile + fixed memory (for InfiniBand) + static graph + asynchronous NCCL (w/ compiled\_autograd)**
  - **No compile** also makes **asynchronous NCCL** (can beat Compile + fixed memory (for InfiniBand) in some cases)
  - **Compile + recompile** make **training slower than no compile**, even than single-node accum. grad.
  - **Gradient accumulation** may be a good option

**이후는 시간 남으면 함.**

## Overview

# Computation Time

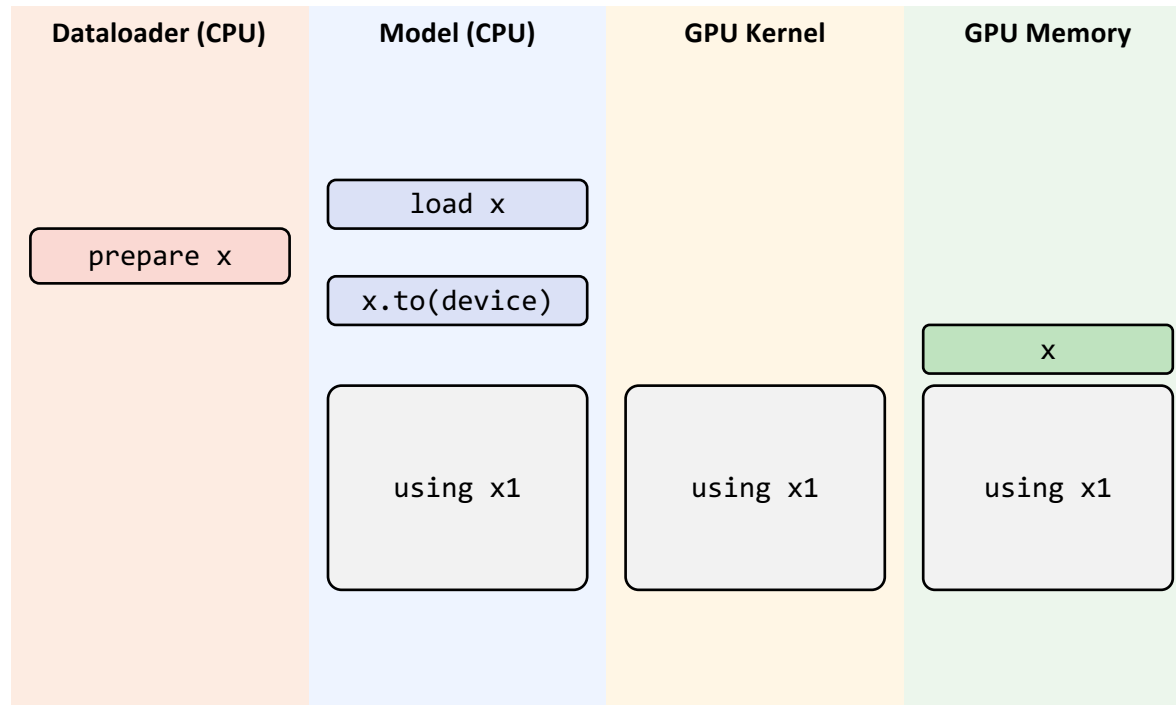


How can we optimize these?



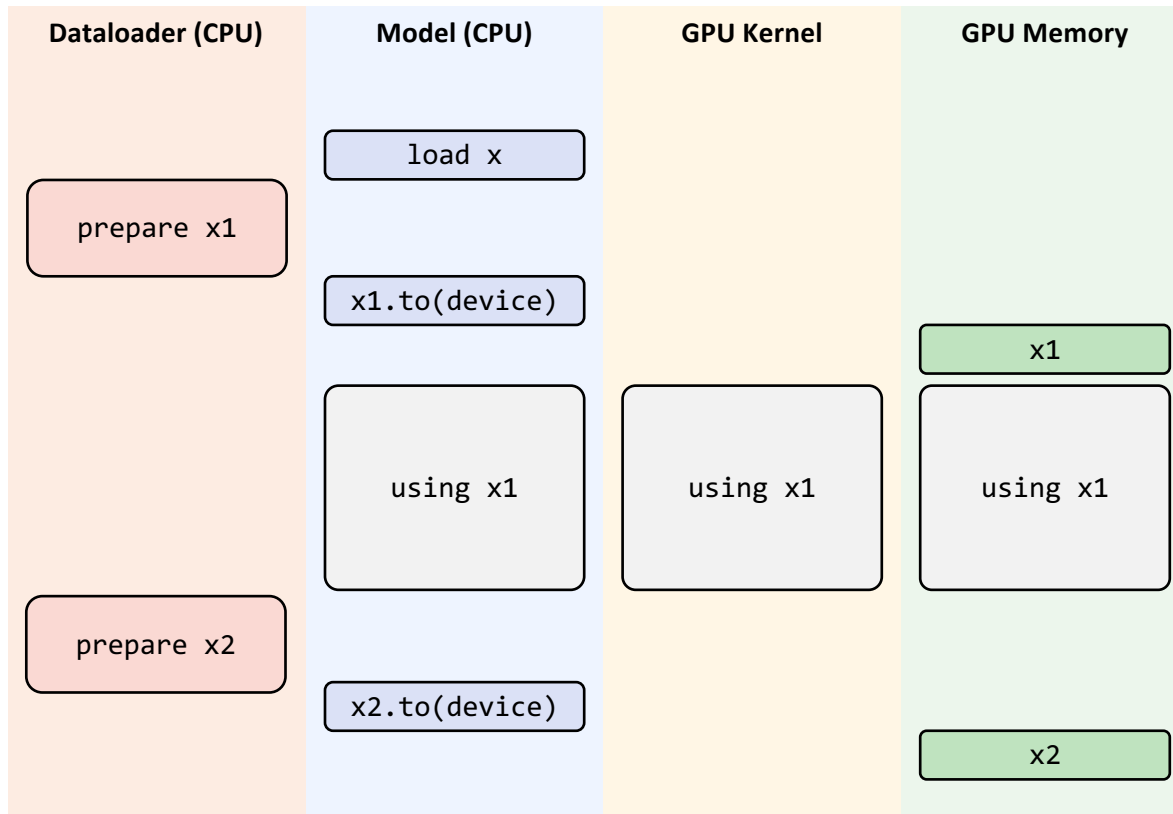
- **Forward and backward computation** time matters in general
- **Communication** overhead matters in **multi-GPU training**
- **Fetch to GPU** time matters when **probing high-dimensional representations** (e.g., pair reps)

## Disk to GPU Data Fetch



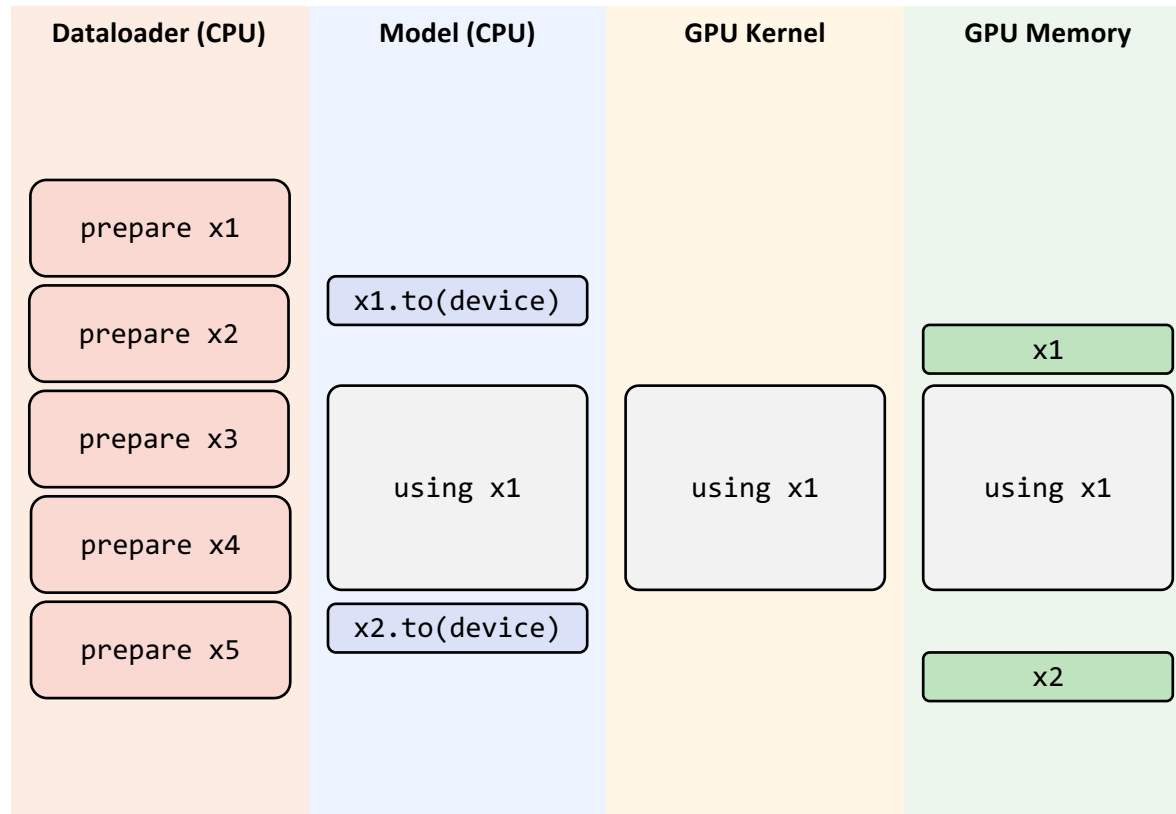
- Conventional case

## Situation: Disk-to-RAM Overhead



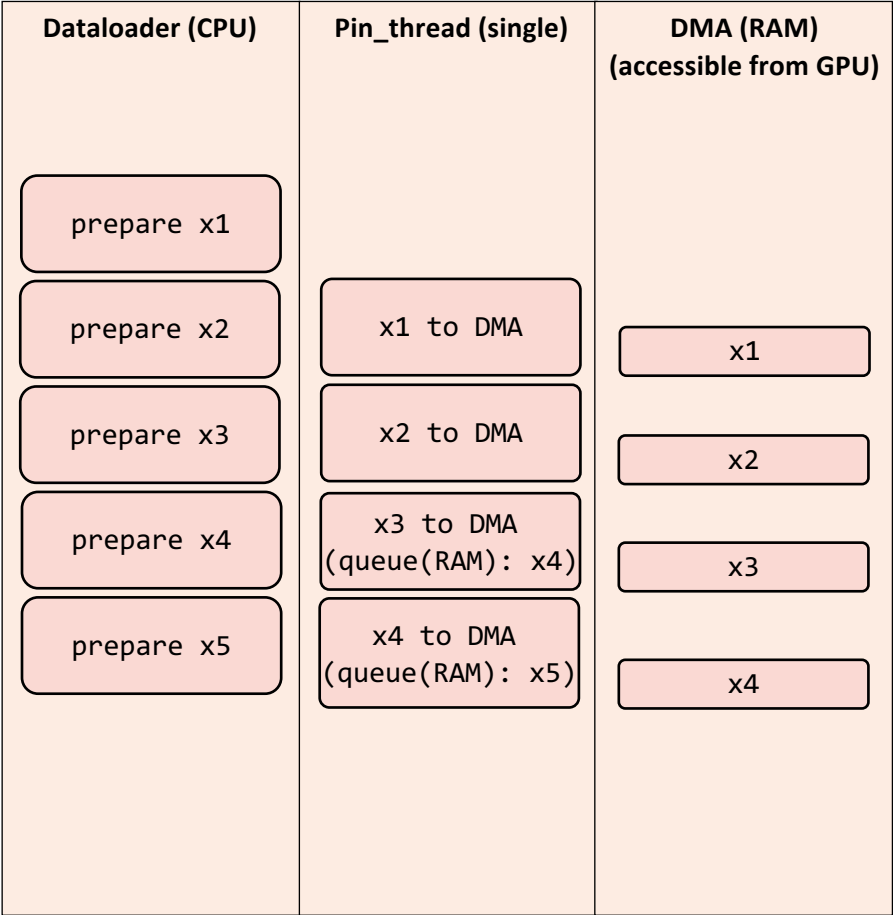
- Large-volume data or pre-processing

## Optimization: Asynchronous Disk-to-RAM

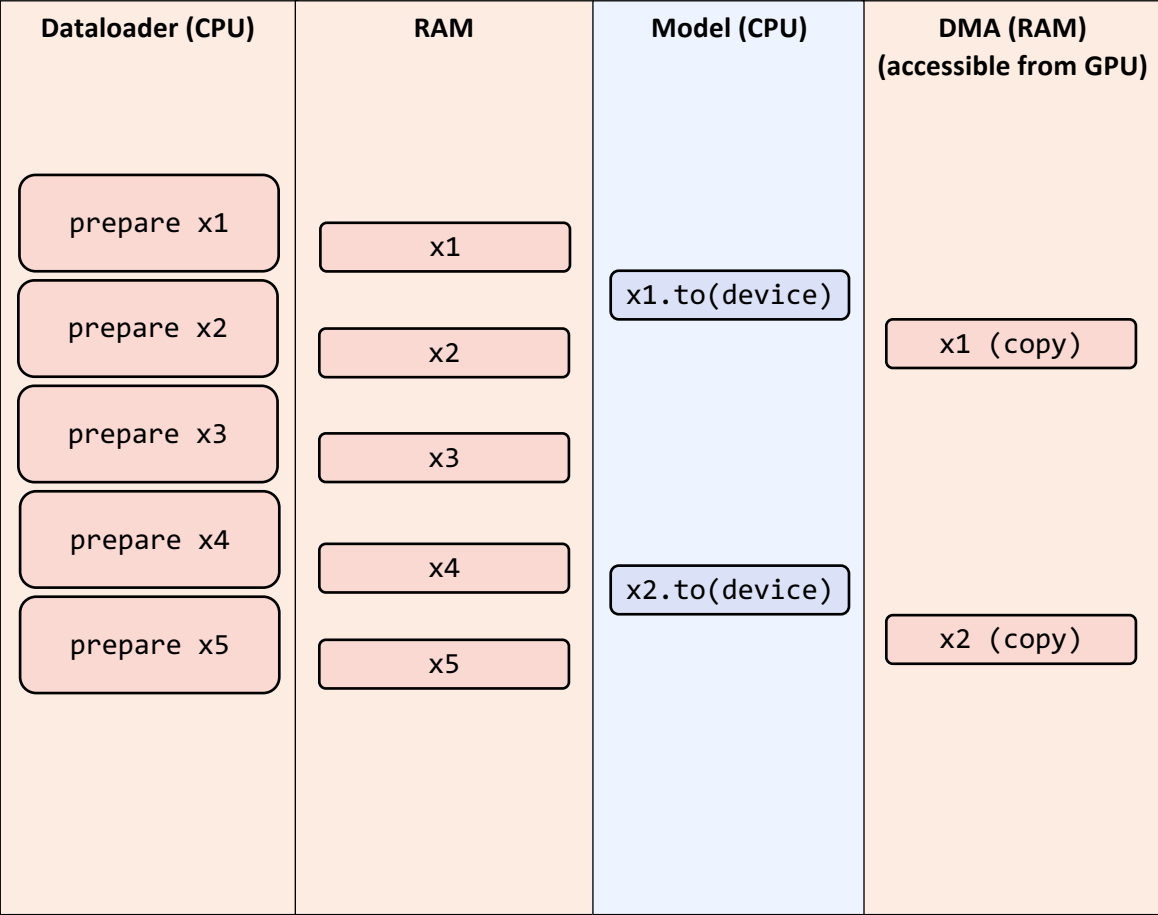


- Num\_workers and prefetch\_factor

# Details in Asynchronous Disk-to-RAM



pin\_memory=True



pin\_memory=False



## Details in Asynchronous Disk-to-RAM

- RAM bandwidth is limited. Increasing num\_workers or prefetch\_factor cannot exceed this

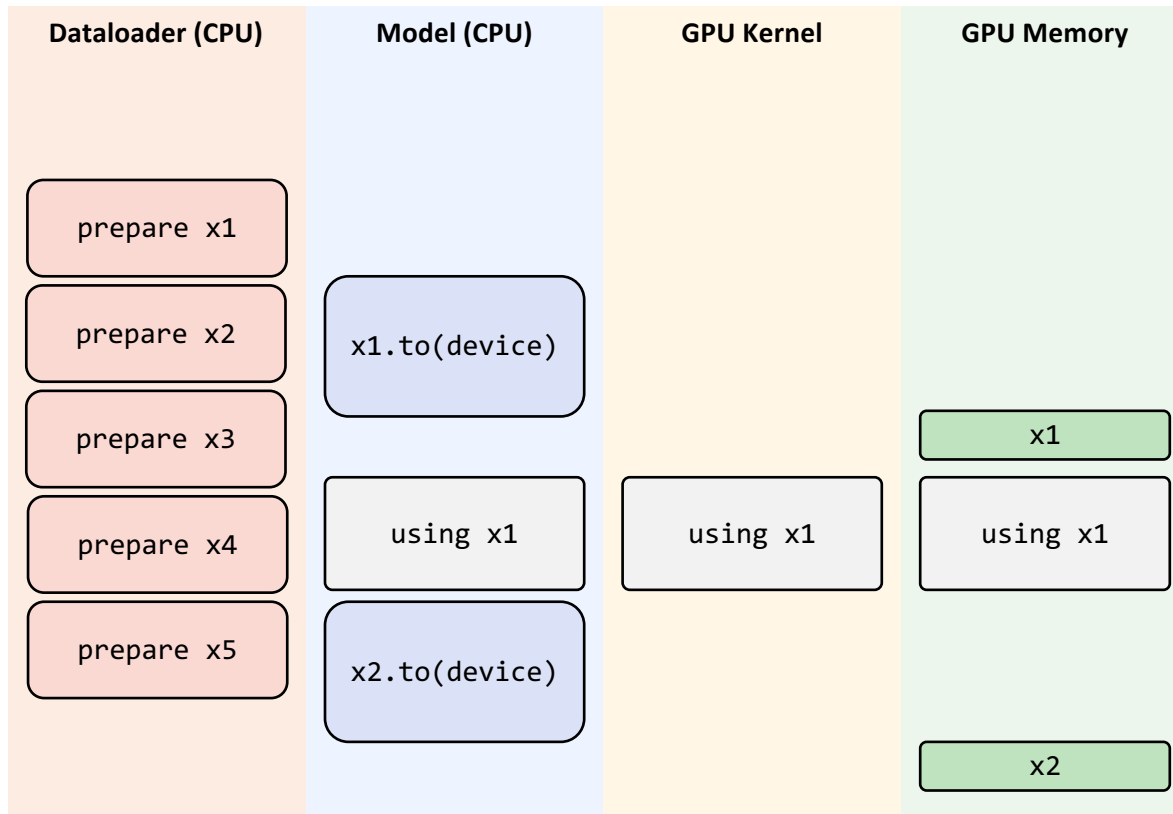
### **pin\_memory = True**

- Many workers write to a **queue (RAM) in parallel**
- A **single pin\_memory\_thread** writes to **DMA while sharing RAM bandwidth** with workers
  - ➔ **Larger num\_workers or prefetch\_factor** can make a **single pin\_memory\_thread slower**

### **pin\_memory = False**

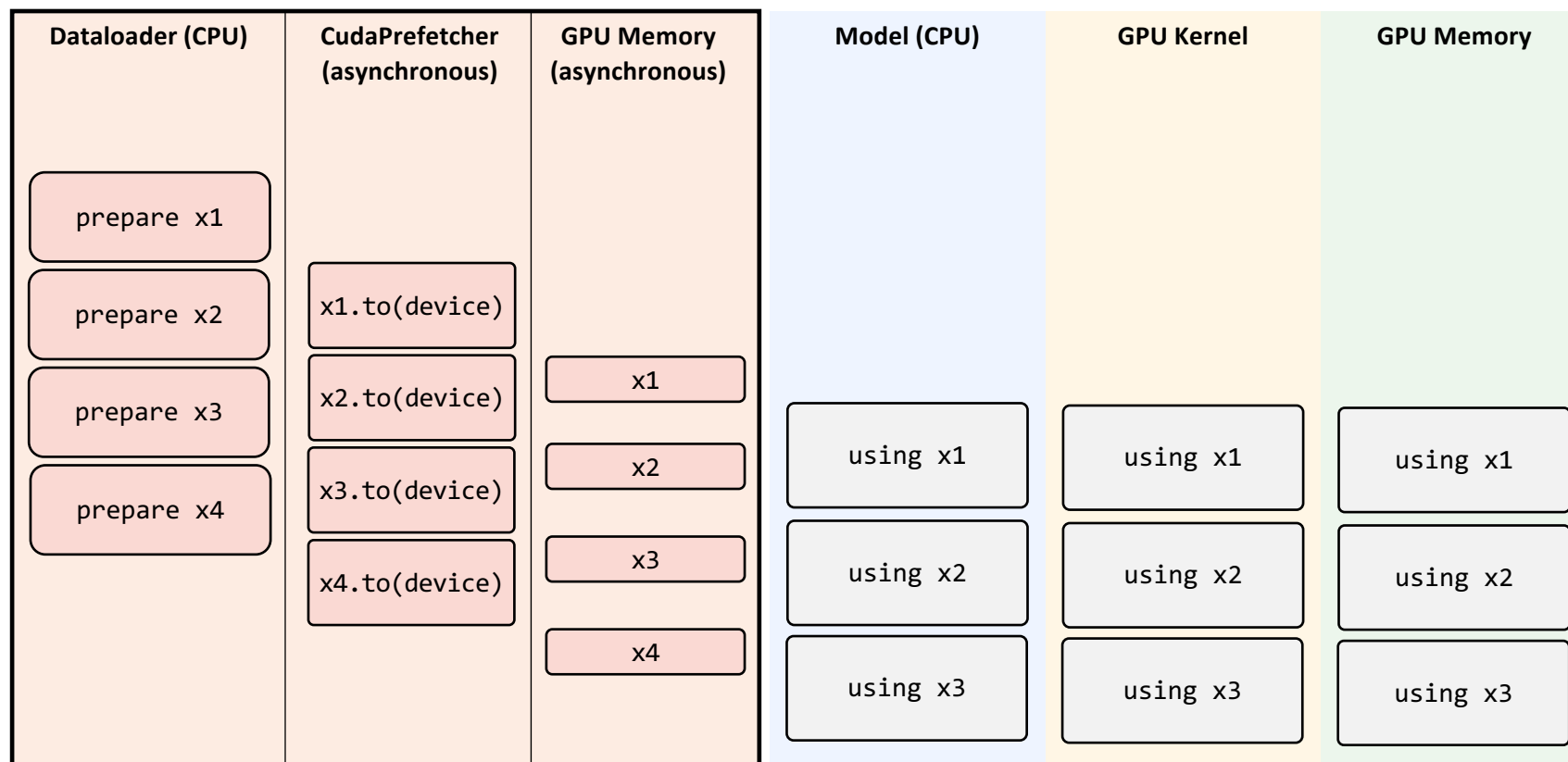
- The .cuda() call **copies data to DMA (with multi thread)** directly **without a single pin\_memory\_thread**
- There is no single-thread bottleneck.

## Situation: RAM-to-GPU Overhead



- Large-volume data (e.g., pair representation)

# Situation: Asynchronous RAM-to-GPU



```

class _CudaPrefetcher:
    def __init__(self, loader, device):
        self.loader = loader
        self.device = device
        self.stream = torch.cuda.Stream(device=device)

    def __len__(self):
        return len(self.loader)

    def __iter__(self):
        first = True
        for batch in self.loader:
            with torch.cuda.stream(self.stream):
                batch_on_device = []
                for item in batch:
                    if torch.is_tensor(item):
                        batch_on_device.append(
                            item.to(self.device, non_blocking=True)
                        )
                    else:
                        batch_on_device.append(item)

            if not first:
                cur_stream = torch.cuda.current_stream(self.device)
                cur_stream.wait_stream(self.stream)

            for t in prepared:
                if torch.is_tensor(t):
                    t.record_stream(cur_stream)

            yield prepared

            prepared = batch_on_device
            first = False

        if not first:
            cur_stream = torch.cuda.current_stream(self.device)
            cur_stream.wait_stream(self.stream)
            for t in prepared:
                if torch.is_tensor(t):
                    t.record_stream(cur_stream)
            yield prepared

```

**Thank you**